



UNIVALI

Universidade do Vale do Itajaí
Escola Politécnica

Memória Principal - Sistemas Operacionais

Prof. Michael Douglas C Alves

Gerência de Memória

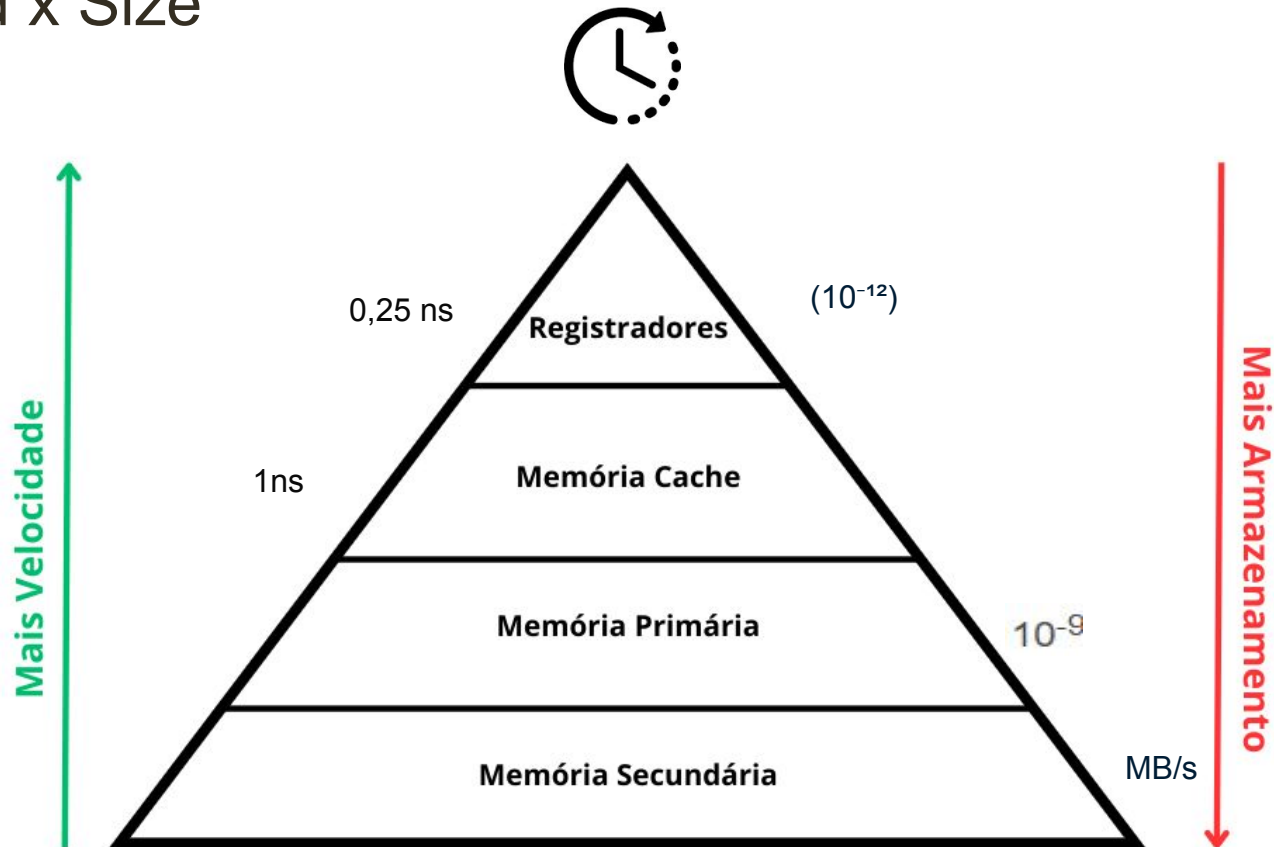
- Conceitos
- Segmentação
- Alocação de memória contínua
Como que é gerenciada
- Paginação
Estrutura da tabela de páginas
- Swapping*

Gerenciamento

- Como a memória principal é gerenciada pelo SO
 - Monoprogramada – um único programa
 - Multiprogramada – divisão da memória em várias partes
- Melhor cenário
 - Memória volátil
 - Velocidade
 - Capacidade de armazenamento
 - Fornecer um pedaço da memória para cada processo.

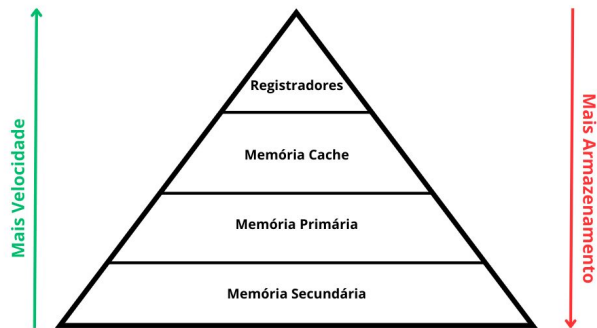
Hierarquia de Memória

- Speed x Size



Hierarquia de Memória

- Speed x Size



Registradores: diretamente dentro da CPU (Processador) do computador e em quantidade muito pequena.

Memória Cache: armazenamento das informações que são utilizadas com frequência pelo processador.

Memória primária: (RAM).

Memória secundária: os elementos de armazenamento físico

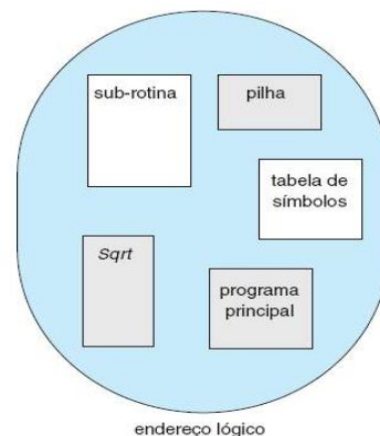
Gerenciamento

- Gerenciador de memória
 - Manter o controle das partes utilizadas e as não utilizadas (Médio prazo)
 - Garantia de recursos aos processos
 - Admitindo ou encerrando
 - Cada processo deve acessar apenas seu espaço de memória.
 - Registradores de base e limite para definir o intervalo válido para acessar a memória
 - deve-se alocar a memória de forma eficiente ao maior número de processos

- O programa deve ser trazido (do disco) para a memória e deve ser transformado em um processo para que ele seja executado
 - A memória principal e os registradores são apenas o armazenamento, que a CPU pode acessar diretamente.
 - Somente o SO pode configurar os registradores (instruções privilegiadas)
- A unidade de memória só vê um fluxo de:
endereços + solicitações de leitura, ou endereço + dados e solicitações de gravação
- O acesso de registrador é feito em um clock da CPU (ou menos)
A memória principal pode levar muitos ciclos, causando uma **Stall** (CPU solicita dados da RAM, ela precisa esperar vários ciclos de clock)
- O cache fica entre a memória principal e os registradores da CPU
 - Proteção da memória necessária para garantir o funcionamento correto (localidade e referência)

Segmentação

- É um esquema de gerenciamento de memória que sustenta a visão do programador.
- A segmentação oferece um mecanismo de memória que mapeia a visão do programador para a memória física real.
- O sistema dispõe de maior liberdade para gerenciar a memória, enquanto o programador usufrui de um ambiente de programação mais produtivo.

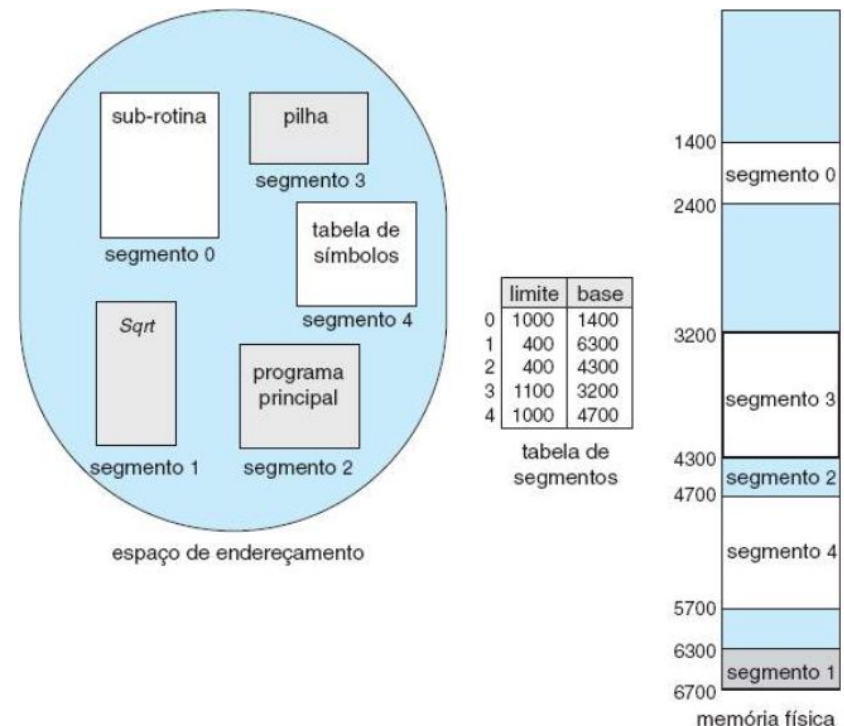


Segmentação

- A alocação vai depender da lógica do programa.
- O mapeamento é feito através das tabelas de mapeamento de segmentos

Normalmente, o compilador constrói automaticamente segmentos que refletem o programa de entrada. Um compilador C pode criar segmentos separados para os elementos a seguir:

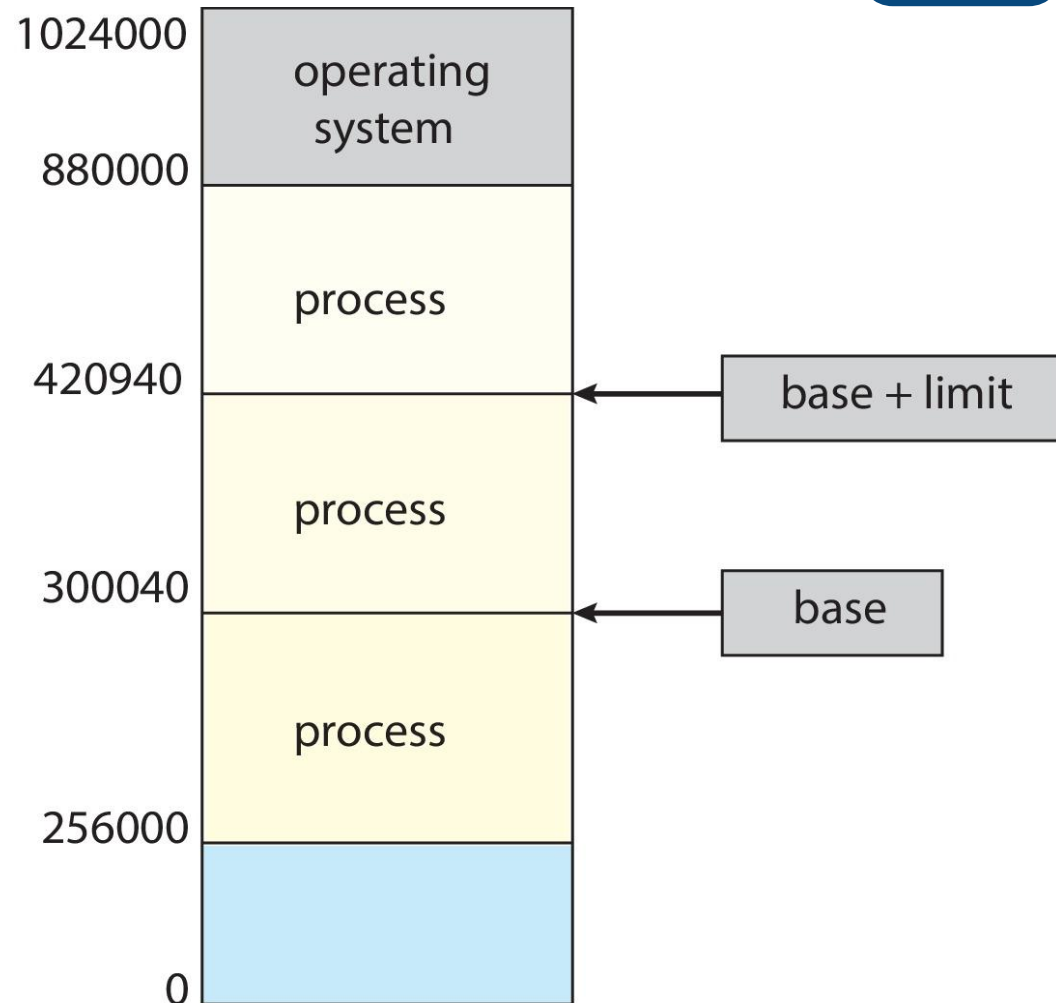
- 1.O código
- 2.Variáveis globais
- 3.O heap a partir do qual a memória é alocada
- 4.As pilhas usadas para cada thread
- 5.A biblioteca C padrão.



Proteção- realocação dinâmica

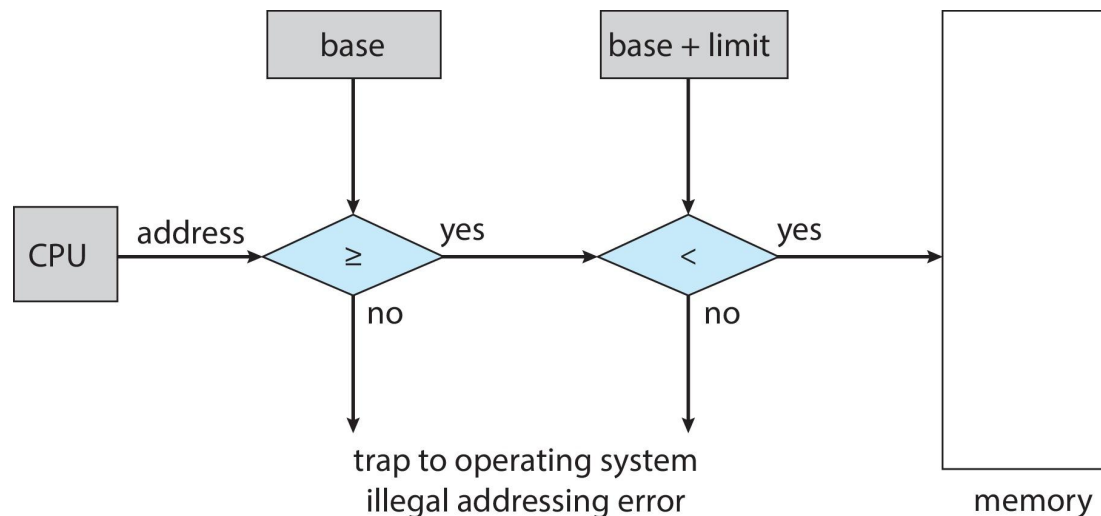
- Necessidade de proteger para que um processo só possa acessar os endereços em seu espaço de endereço

Podemos fornecer essa proteção usando um par de registradores de base e limite para definir o espaço de endereço lógico de um processo



Proteção de endereço de hardware

- A CPU deve verificar cada acesso à memória gerado no modo de usuário, para ter certeza de que está entre base e limite para esse usuário



- As instruções para carregar os registradores de base e limite são privilegiadas (SO-Assembly)

Linkagem de endereço

- Programas em disco aguardam na fila de entrada para serem carregados na memória.
 - Sem configuração, carregam no endereço 0000.
 - Não é ideal que processos de usuário carreguem no endereço físico 0000.
 - **Endereços se transformam:**
 - Pelo programador temos o código-fonte: simbólicos (cout).
 - Código compilado: endereços relocáveis (ex: "14 bytes do início do módulo").
 - **Linker/loader:** Vincula os endereços relocáveis a endereços absolutos/físicos (ex: transforma uma posição relativa em um endereço real como 74000→74014).
 - Endereços são traduzidos de simbólicos → relocáveis → físicos
- Momento de compilação, carregamento e execução.

Linkagem - Binding

- Processo de associar instruções e dados aos seus endereços reais na memória
 - A vinculação de instruções e dados a endereços de memória pode ocorrer em três estágios diferentes:
 - 1- Tempo de compilação: Ocorre quando o compilador sabe exatamente em que endereço o programa residirá na memória. Se o local mudar futuramente, o programa precisará ser recompilado.
 - Para mudar o programa de lugar, era necessário alterar o código-fonte e recompilar tudo para o novo endereço.

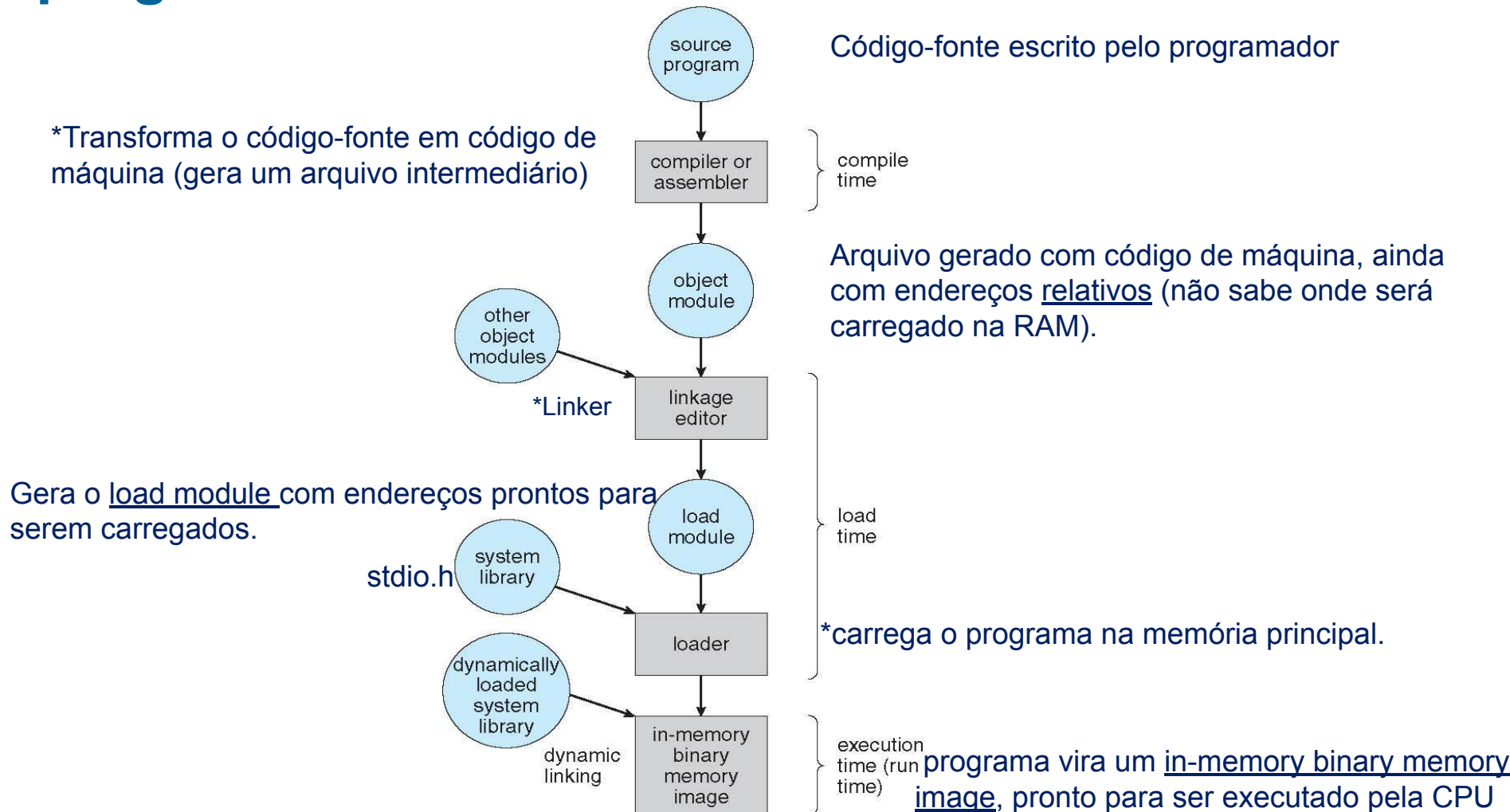
Linkagem - Binding

- Processo de associar instruções e dados aos seus endereços reais na memória
 - A vinculação de instruções e dados a endereços de memória pode ocorrer em três estágios diferentes:
 - 2- Tempo de carregamento: Caso o endereço não seja conhecido, o compilador tem que gerar um código relocável
 - Funcionamento das DLLs (Dynamic Link Libraries) no Windows, ou arquivos .so (Shared Objects) no Linux
 - O Loader (carregador) do Windows lê o `.exe` e vê que ele precisa da `math.dll`.
 - O Windows carrega a `math.dll`.

Linkagem - Binding

- Processo de associar instruções e dados aos seus endereços reais na memória
 - A vinculação de instruções e dados a endereços de memória pode ocorrer em três estágios diferentes:
 - 3- Tempo de execução: o processo pode ser movido durante sua execução de um segmento de memória para outro, ou seja o processo muda deposição na memoria durante a própria execução. (MMU)
 - Precisa de suporte de hardware para mapas de endereços (por exemplo, registradores de base e limite)

Processamento em várias etapas de um programa de usuário

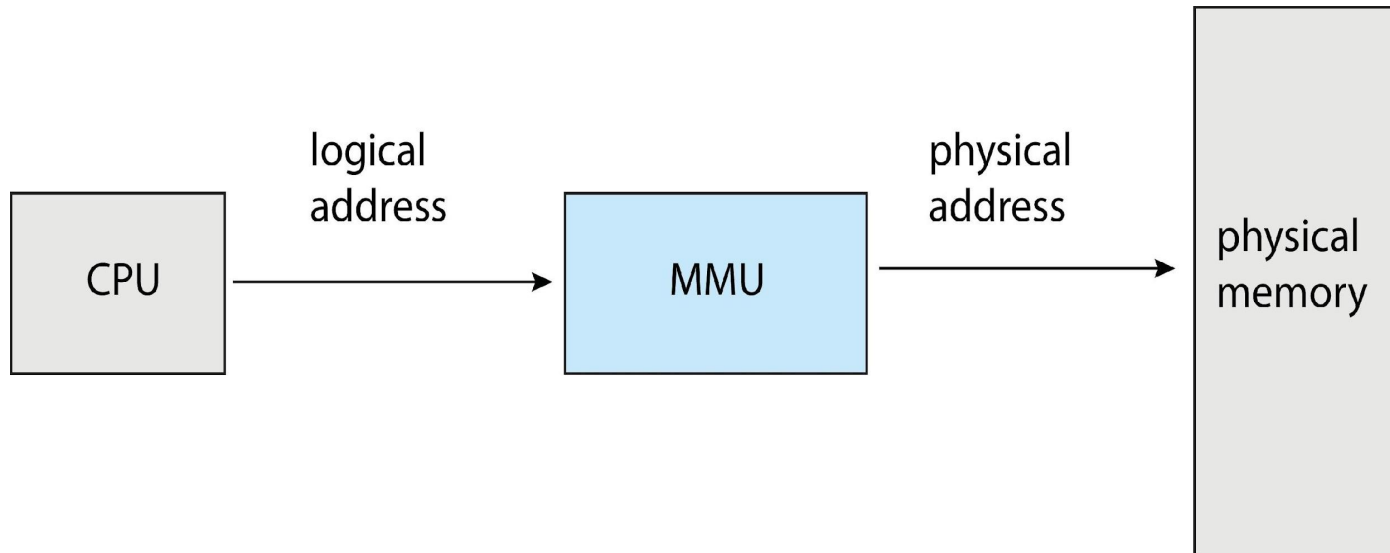


Espaço de endereço lógico vs. físico

- O conceito de separação entre espaço de endereçamento lógico e físico é fundamental para o gerenciamento adequado da memória.
 - Endereço lógico – gerado pela CPU; também conhecido como endereço virtual
 - Endereço físico – endereço visto pela unidade de memória
 - Os endereços lógicos e físicos são os mesmos em esquemas de vinculação de endereço em **tempo de compilação** e **tempo de carregamento**;
 - Os endereços lógicos (virtuais) e físicos diferem no esquema de vinculação de endereço em **tempo de execução**
- **Espaço de endereço lógico**: é o conjunto de todos os endereços lógicos gerados por um programa.
Espaço de endereço físico é o conjunto de todos os endereços físicos gerados por um programa.

Memory-Management Unit (MMU)

- Dispositivo ou componente de hardware que, em tempo de execução, mapeia o endereço virtual para o físico.



Memory-Management Unit (MMU)

- Considere um esquema simples: uma generalização do esquema de registrador de base.

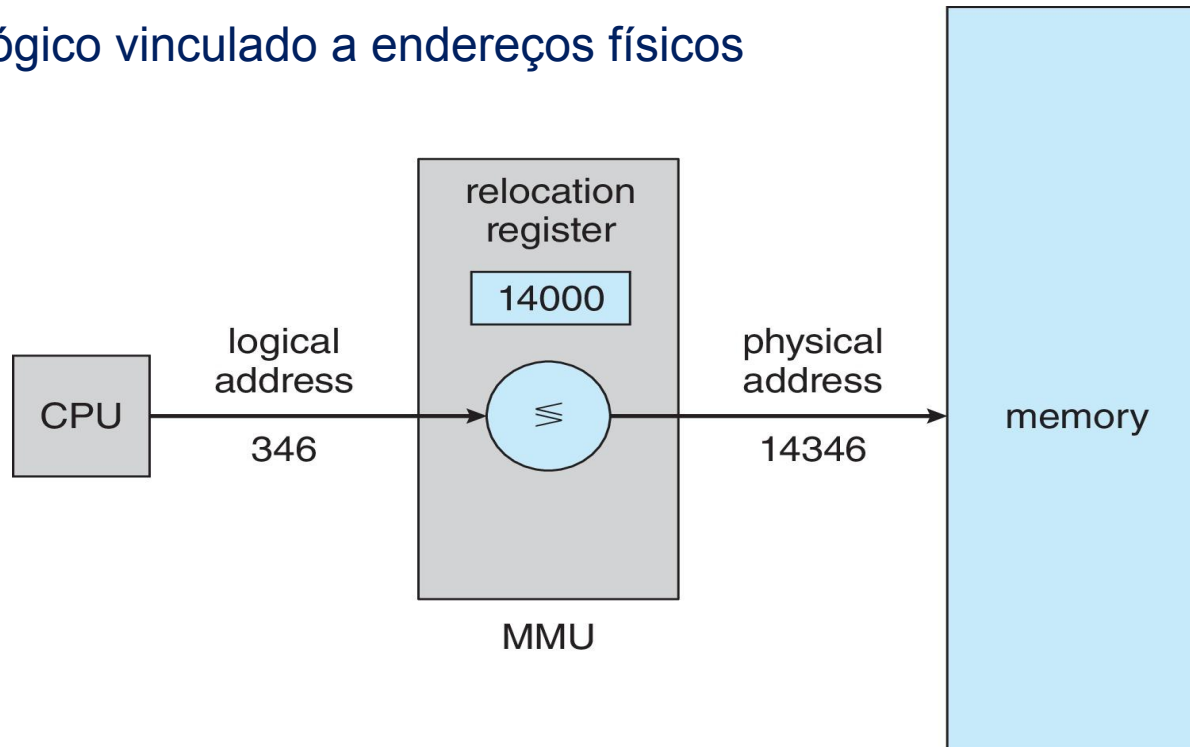
O registrador base agora chamado de registrador de *offset*.

O valor do registrador de offset é adicionado a todo endereço gerado por um processo de usuário no momento em que é enviado para a memória.

O programa de usuário lida com endereços lógicos; ele nunca vê os endereços físicos reais.

Memory-Management Unit (MMU)

Endereço lógico vinculado a endereços físicos



A linkagem em tempo de execução ocorre quando é feita referência ao local na memória.

Carregamento/Carga dinâmico(a)

- Técnica que permite carregar partes do processo do disco para a memória dinamicamente.
- A rotina não é carregada até que seja chamada
 - Melhor utilização do espaço de memória
- Todas as rotinas mantidas em disco em formato de carga realocável:
 - Útil quando grandes quantidades de código são necessárias para lidar com casos que ocorrem com pouca frequência
- Nenhum suporte especial do sistema operacional é necessário
 - Implementado através do design do programa: ex `dlopen()` e `dlclose()` (carrega e descarrega lib)
O sistema operacional pode ajudar fornecendo libs para implementar o carregamento dinâmico

Métodos de Vinculação estática x dinâmica de end. na memória

- Vinculação estática – Código de bibliotecas externas são incorporados diretamente em arquivo executável. (Imagem do programa).
 - O programa carrega todas as bibliotecas necessárias durante a compilação, resultando num executável independente.
 - Programa não depende de bibliotecas externas serem carregadas durante a execução.
- Vinculação dinâmica – durante a execução.
- Fragmento de código no SO: stub, é usado para localizar a rotina apropriada da biblioteca. Chama o loader do SO para buscar a função na biblioteca compartilhada (.so ou .dll).
- O Stub substitui a si mesmo como endereço da rotina e a executa. O SO verifica se a rotina está no endereço de memória dos processos;
 - Se não estiver no espaço de endereço, adicione ao espaço de endereço

Métodos de Ligação dinâmica

- A vinculação dinâmica é particularmente útil para bibliotecas.
 - Sistema também conhecido como bibliotecas compartilhadas: várias aplicações compartilhem da mesma
 - Considere a aplicabilidade para corrigir bibliotecas do sistema : bibliotecas vinculadas dinamicamente podem ser atualizadas sem que os aplicativos que as usam precisem ser recompilado.
 - Bibliotecas de terceiros
 - Pode não ser possível.

Alocação contígua de memória

- Recurso limitado, deve alocar de forma eficiente.

A alocação contígua é um dos primeiros métodos:

- Cada processo ocupa uma área contínua da memória (sem divisões externas).

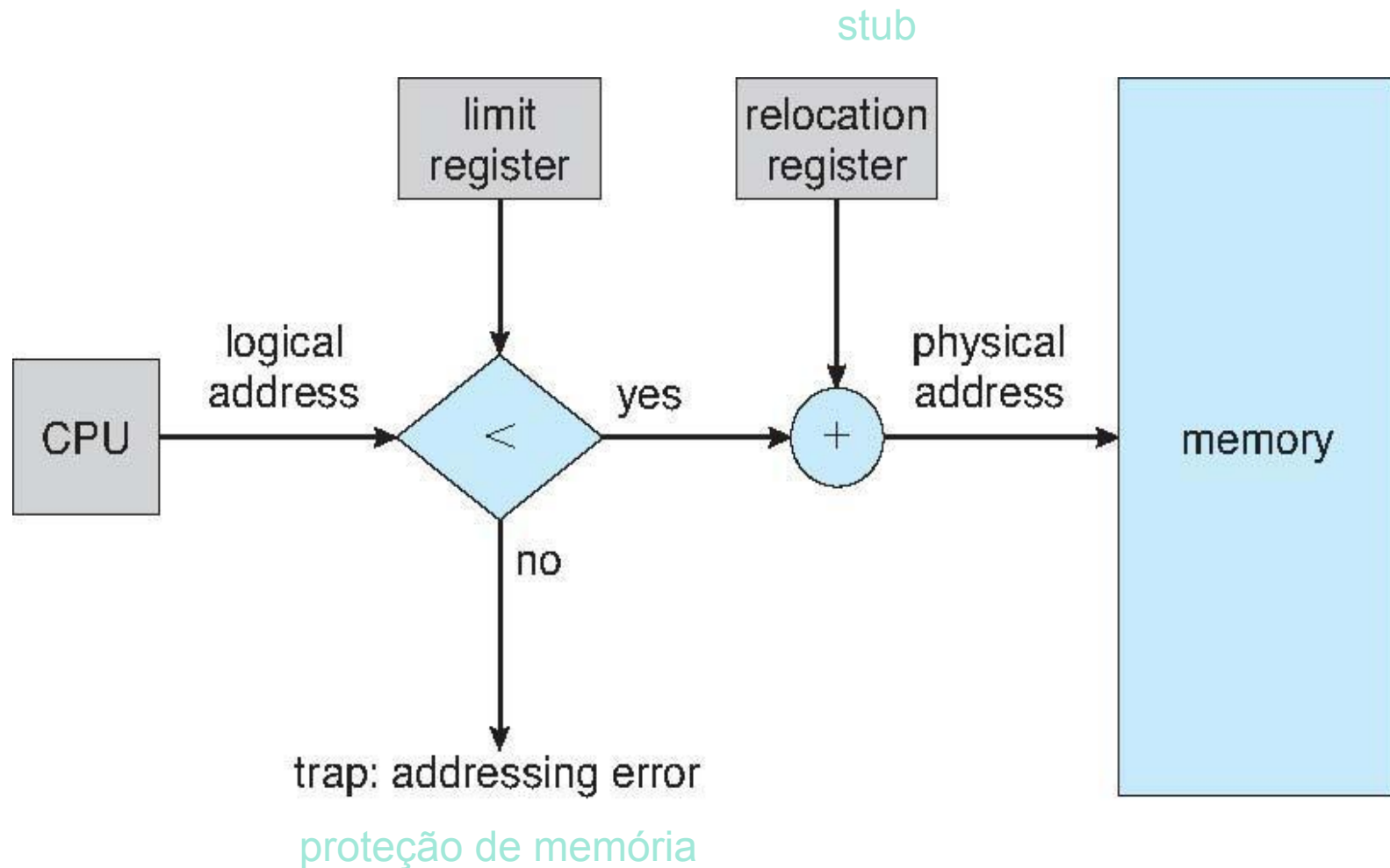
A organização da memória é dividida em duas partições:

- Parte baixa: Reservada para o Sistema Operacional (SO) e o vetor de interrupções, geralmente ocupando endereços de memória menores.
- Parte alta: Destinada aos processos do usuário, localizados em endereços de memória superiores (Programas)

Alocação contígua de memória

- Registradores de realocação usados para proteger os processos do usuário uns dos outros e contra a alteração do código e dos dados do sistema operacional:
 - O registrador base: Armazena o endereço físico inicial (o menor) alocado para o processo.
 - O registrador de limite: Define o tamanho do segmento de memória. Qualquer endereço lógico gerado pelo processo deve ser menor que esse valor para ser considerado válido.
 - MMU mapeia o endereço lógico dinamicamente
 - Pode então permitir ações como o código do kernel sendo transitório e o kernel mudando de tamanho.
 - Kernel não precisa ficar engessado.
 - Sistema operacional ganha flexibilidade para mover, aumentar ou diminuir o kernel na memória.

Suporte de hardware para realocação e registrador de limite



Partição variável

- Alocação de várias partições

- Grau de multiprogramação limitado pelo número de partições;

Tamanhos de partição variável para eficiência (dimensionados de acordo com as necessidades de um determinado processo);

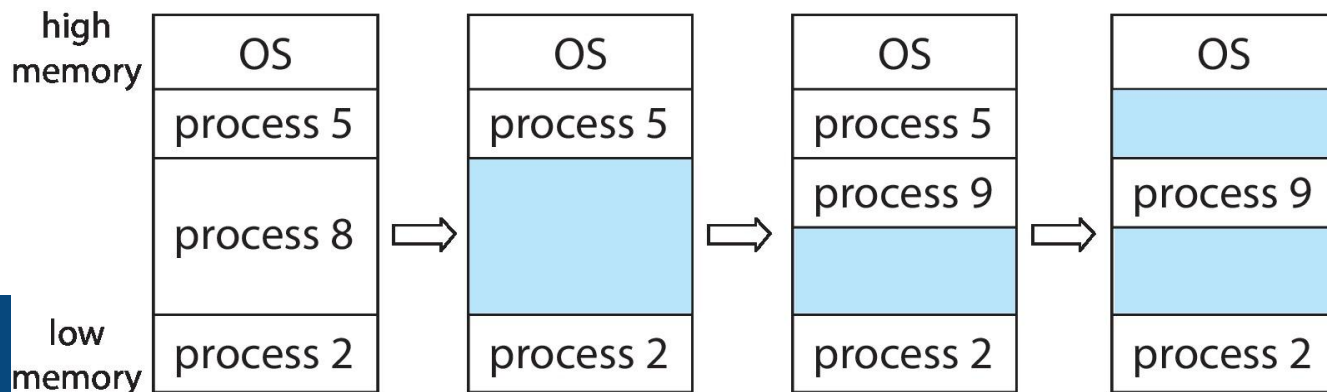
Buraco – bloco de memória disponível; buracos de vários tamanhos estão espalhados por toda a memória;

Quando um processo chega, ele recebe memória de um parte grande o suficiente para acomodá-lo

Processo de saída libera sua partição e partições livres adjacentes combinada;

O sistema operacional mantém informações sobre:

a) partições alocadas b) partições livres (buracos)



Problema de armazenamento com alocação dinâmica

- Como satisfazer um pedido de tamanho n de uma lista de buracos (memória) livres?
 - First-fit: Aloque a primeira lacuna disponível e tamanho suficiente (deixa pequenos buracos)
 - Best-fit: Aloque a menor lacuna com a menor sobra; deve pesquisar a lista inteira, a menos que ordenado por tamanho (lento, varre a memória inteira)
 - Resulta em um o menor buraco restante
 - Worst-fit: Alocar o maior espaço; também deve pesquisar a lista inteira
 - Produz o maior espaço não alocado
- First-fit e best-fit são melhores do que worst-fit em termos de velocidade e utilização do armazenamento

Fragmentação

- **Fragmentação externa** – Existe espaço total de memória para satisfazer uma solicitação, mas não é contíguo.
- **Fragmentação interna** – A memória alocada pode ser um pouco maior do que a memória solicitada;

A primeira análise de ajuste revela que, dados os N blocos alocados, 50% dos N blocos perdidos para a fragmentação

- 1/3 pode ser inutilizável -> regra de 50%

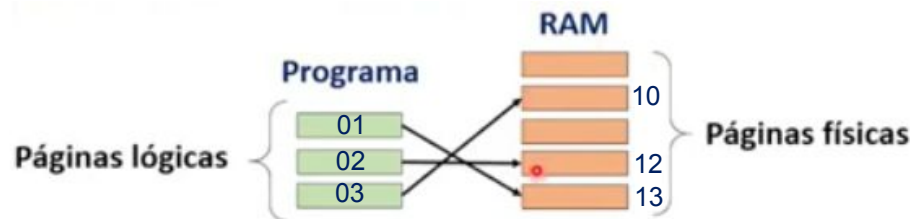
Como reduzir???

Fragmentação

- Reduzir a fragmentação externa por compactação
 - Embaralhe o conteúdo da memória para colocar toda a memória livre junta em um grande bloco!
 - A compactação só é possível se a realocação for dinâmica e for feita no momento da execução
 - Problema de E/S (por exemplo, lendo ou gravando no disco), não pode ser movido.
Operação trava na memória enquanto ela está envolvida na E/S
 - Fazer E/S somente em buffers do sistema operacional: pois os buffers são gerenciados separadamente e podem ser realocados de forma segura
- Obs: Agora considere que o armazenamento de discos e backup tem os mesmos problemas de fragmentação.

Paginação - Conceito

- Técnica de gerenciamento de memória, com divisões na memória física e nos processos.
- Cada página de um processo pode ser colocada em qualquer quadro livre da memória.
- As páginas não precisam estar juntas na memória física, melhora o aproveitamento da RAM



Memória Lóg	Memória Fis
01	13
02	12
03	10

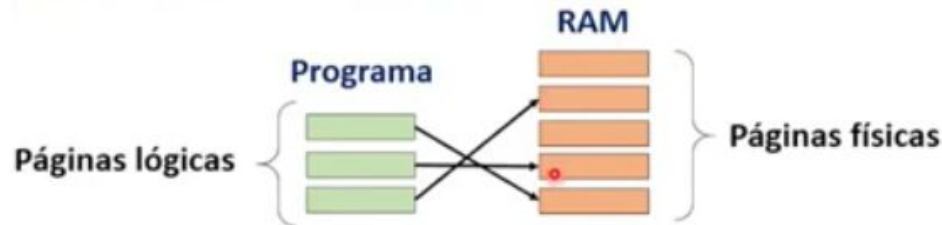
Paginação

- A compactação reorganiza a memória em tempo de execução para juntar todos os blocos livres em um único espaço contínuo.
 - Evita a fragmentação externa;
 - Evita o problema de blocos de memória de tamanho variável;
 - Ajuda a rodar processos maiores do que a memória disponível;
- Divide a memória lógica de um processo em blocos de tamanho fixo chamados páginas e a memória física (RAM) em blocos de mesmo tamanho chamados frames ou quadros.
 - Tamanho é potência de 2, entre 512 bytes e 1GB (4KB - 16 Mbytes).

Paginação

Características:

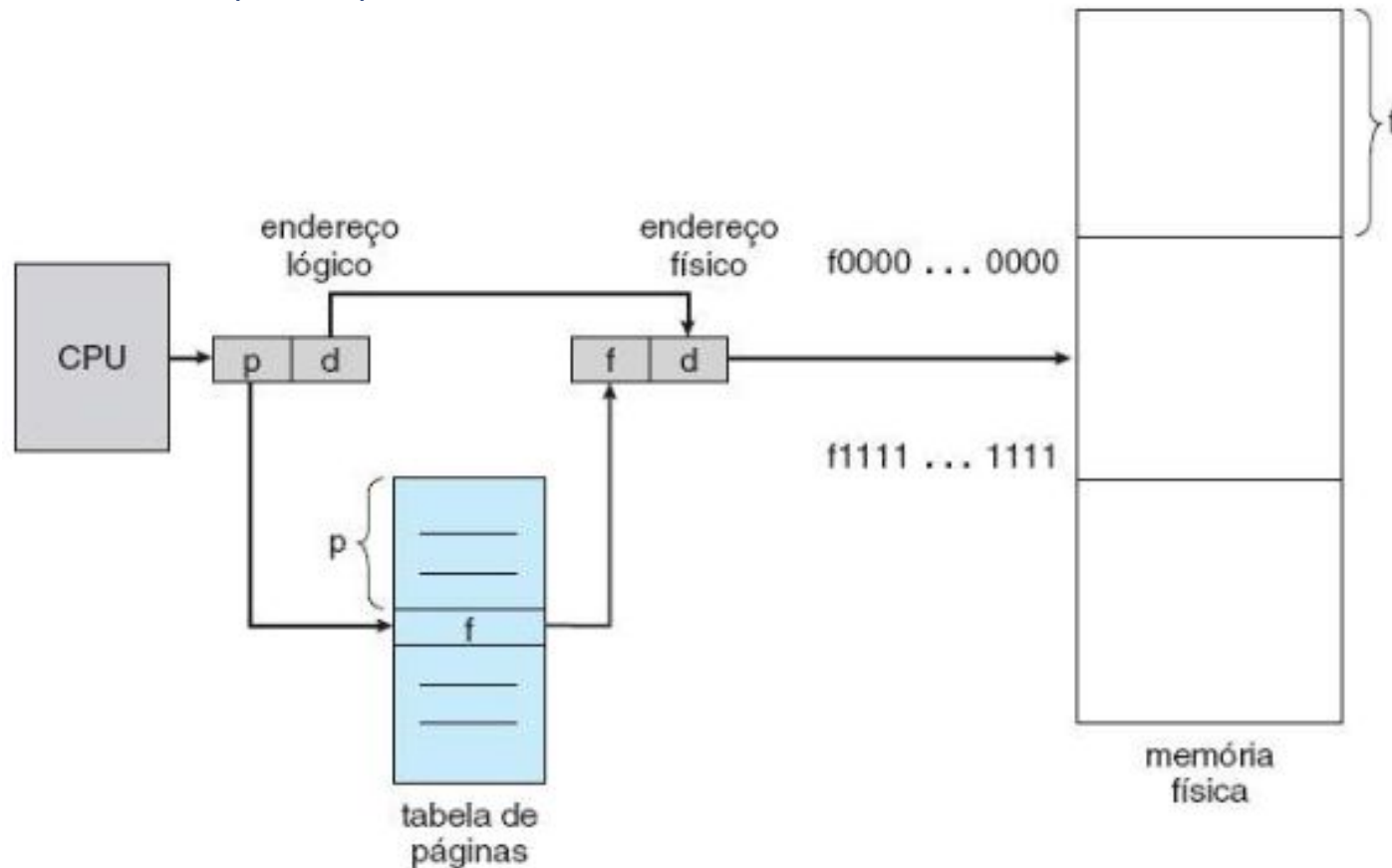
- Acompanhar todos os quadros livres.
- Para executar um programa de tamanho N páginas, precisa encontrar N quadros livres e carregar o programa.
- Configurar uma tabela de páginas para converter endereços lógicos em físicos



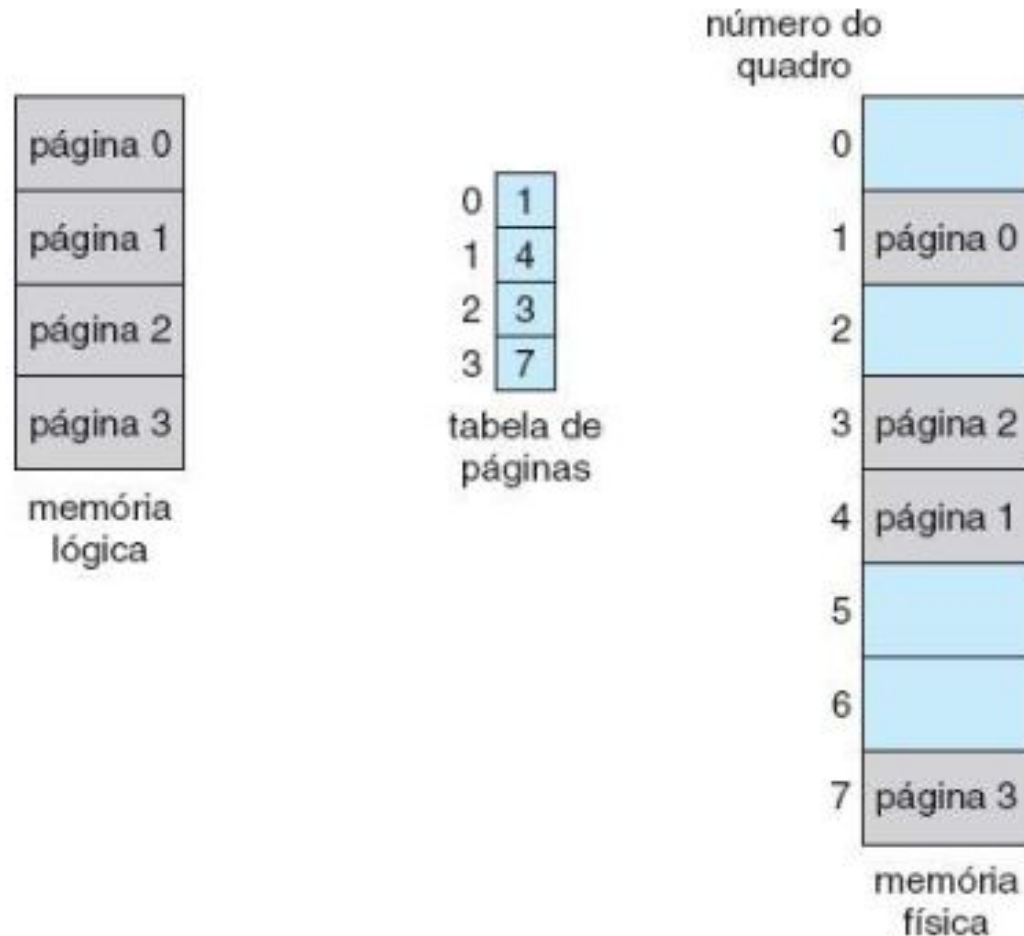
- O armazenamento de backup também é dividido em páginas, mas ainda temos fragmentação interna

Hardware de paginação

(p) – Índice da tabela de página
(d) - Deslocamento (offset)



Modelo de Paginação das Memória Lógica e Física



Exemplo de paginação

End. lógico

- Offset 2 bits
- Usando um tamanho de página de 4 bytes
- Memória física de 32 bytes (8 páginas)

0	a
1	b
2	c
3	d
4	e
5	f
6	g
7	h
8	i
9	j
10	k
11	l
12	m
13	n
14	o
15	p

memória lógica

0	5
1	6
2	1
3	2

tabela de páginas

0	
4	i
	j
	k
	l
8	m
	n
	o
	p
12	
16	
20	a
	b
	c
	d
24	e
	f
	g
	h
28	

memória física

$$\text{End_físico} = \text{TabelaPaginas}[p].\text{Célula} * \text{TamPag} + d$$

Paginação – Calculando a fragmentação interna

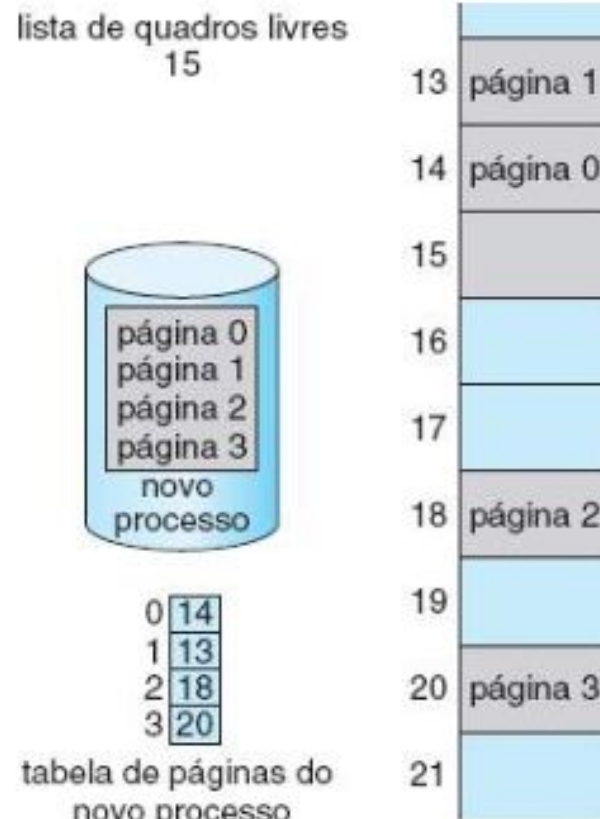
- Tamanho da página = 2.048 bytes
Tamanho do processo = 72.766 bytes
35 páginas + 1.086 bytes
Fragmentação interna de $2.048 - 1.086 = 962$ bytes
Fragmentação no pior caso = 1 quadro – 1 byte
 - Nesse exemplo, 2.047 bytes
- Em média, fragmentação interna perde por processo = 50% do tamanho da página.
- Tamanhos de quadro estão pequenos o suficiente?
 - Quadros muito pequenos: diminui a fragmentação, mas exigem muitas entradas na tabela de páginas (o que consome memória)
Os tamanhos das páginas crescem ao longo do tempo

Quadros (Frames) Livres



(a)

Antes da
alocação



(b)

Após a
alocação

Implementação da tabela de páginas

- A tabela de páginas é mantida na memória principal
 - Page-table base register (PTBR) aponta para a tabela de páginas.
 - Page-table length register (PTLR) indica o tamanho da tabela de páginas.
- Nesse esquema, todo acesso a dados/instruções requer dois acessos à memória
 - Um para a tabela de páginas e outro para os dados/instrução
- O problema de dois acessos a memórias pode ser resolvido com o uso de um cache de hardware especial de pesquisa rápida chamado *Translation Look-Aside Buffers (TLBs)* (também chamada de *memória associativa*)

Translation Look-Aside Buffer

- Memória especial: armazena temporariamente as traduções de endereços virtuais para físicos.
-
- TLBs são caches pequenas de hardware usadas para acelerar a tradução de endereços virtuais para físicos
- Alguns TLBs armazenam identificadores de espaço de endereço:
 - (Address Space Identifiers)- Diferenciar os processos.
 - Permite que o sistema proteja a memória de cada processo e evita que o cache precise ser "limpo" (esvaziado) toda vez que há uma troca de tarefa (troca de contexto).

Translation Look-Aside Buffer

- TLBs tipicamente pequenas (64 a 1024 entradas)
- Em uma miss de TLB, o valor é carregado na TLB para acesso mais rápido na próxima vez
 - Quando a “tradução” não é encontrada no cache, o valor é buscado na tabela de páginas (na memória principal) e carregado no TLB para que o acesso seja mais rápido nas próximas vezes.
 - Algumas políticas de substituição devem ser consideradas, por exemplo: o sistema precisa de regras para decidir qual tradução antiga remover para dar lugar a uma nova.

Como funcionam as Políticas de Substituição?

Como o TLB é muito pequeno (64 a 1024 entradas), ele enche rápido. Para colocar uma tradução nova, precisamos expulsar uma antiga.

- Memória associativa – pesquisa paralela

aponta para o início da tabela de páginas

Page #	Frame #

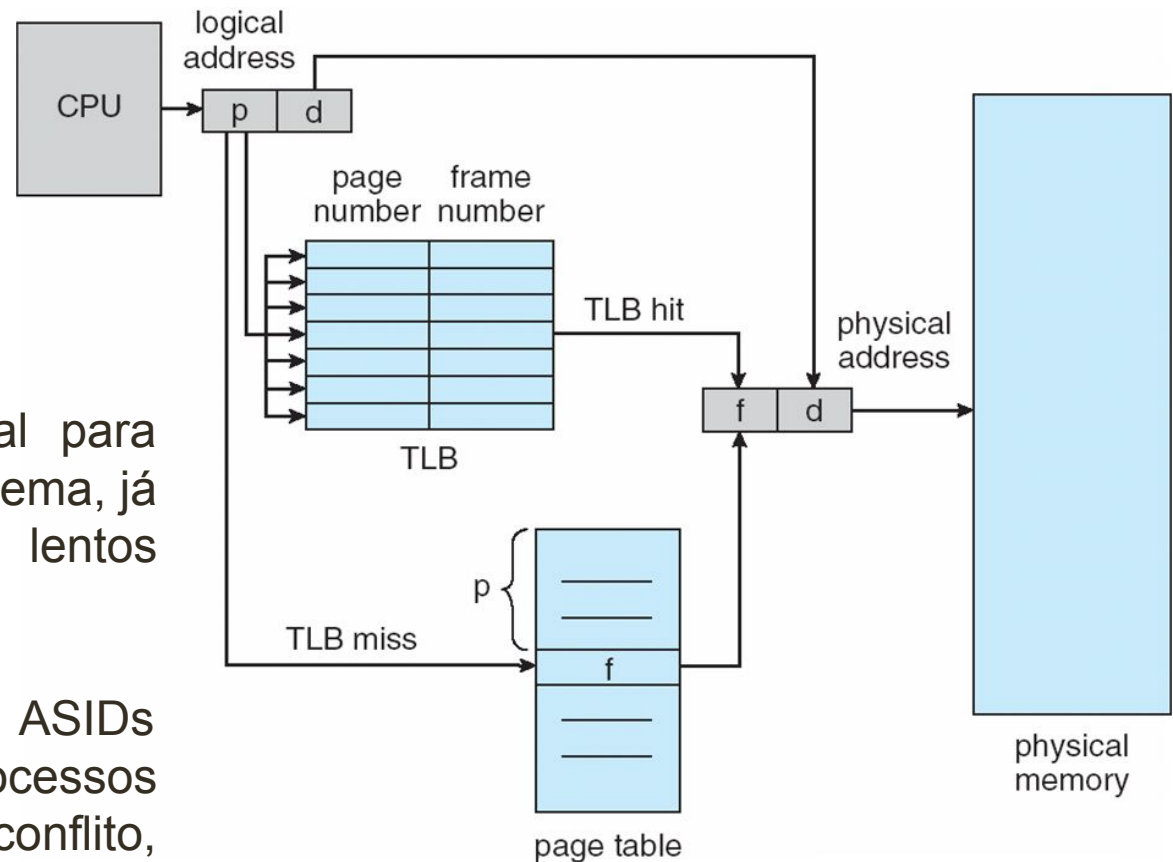
Tamanho da tabela de páginas

- Tradução de endereço (p, d)
 - Se 'p' estiver no registrador associativo, obtenha o quadro # da memória física
 - Caso contrário, obtenha o quadro # da tabela de páginas na memória (dois acessos)

Hardware de paginação com TLB

O uso de TLBs é essencial para aumentar a velocidade do sistema, já que acessos à RAM são lentos comparados à TLB.

Em sistemas multitarefa, os ASIDs permitem que múltiplos processos usem a TLB sem conflito, melhorando ainda mais a eficiência



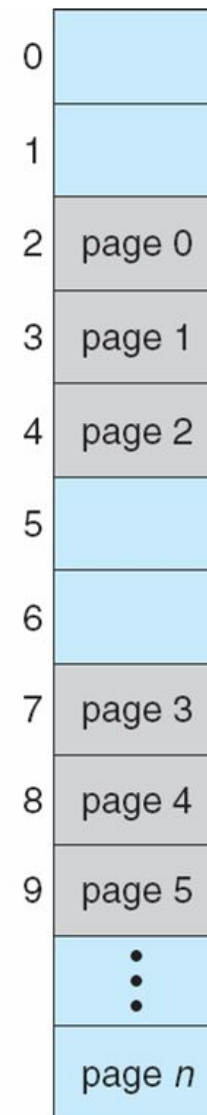
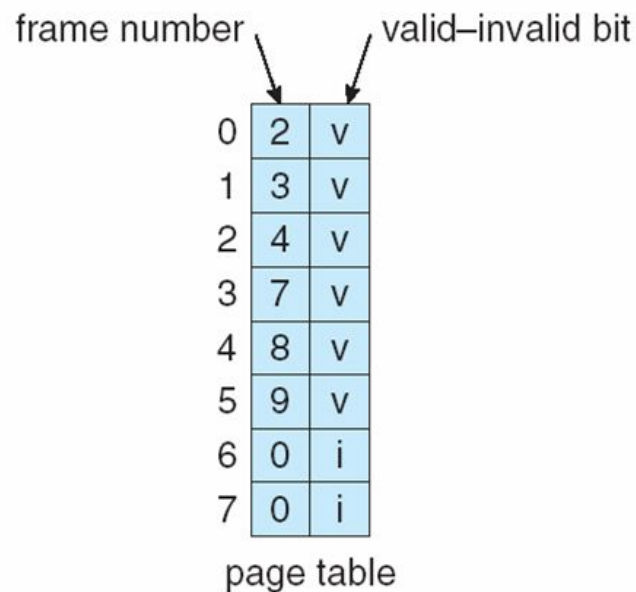
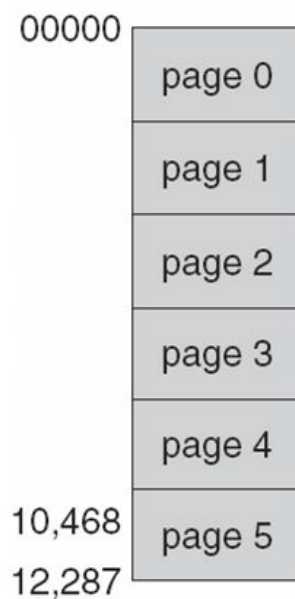
Tempo de Acesso Efetivo

- Taxa de acertos – porcentagem de vezes que um número de página é encontrado na TLB?
- Uma taxa de acerto de 80% significa que encontramos o número da página desejada na TLB 80% das vezes.
- Suponha que 10 nanossegundos para acessar a memória:
 - Se encontrarmos a página desejada na TLB, um acesso à memória mapeada leva 10 ns;
Caso contrário, precisamos de dois acessos à memória, por isso é 20 ns;
- Tempo de Acesso Efetivo (EAT)
$$\text{EAT} = (0,80 \times 10) + 0,20 \times 20 = 12 \text{ nanossegundos}$$
implicando 20% de lentidão no tempo de acesso
- Considere uma taxa de acerto mais realista de 99%
$$\text{EAT} = 0,99 \times 10 + 0,01 \times 20 = 10,1 \text{ ns}$$
implicando apenas 1% de desaceleração no tempo de acesso, já que o ideal (TEMPO MÉDIO) é 10ns.

Proteção de memória

- Proteção de memória: implementada associando o bit de proteção a cada quadro para indicar se o acesso somente leitura ou leitura/gravação é permitido
 - Também pode adicionar mais bits para indicar somente execução da página e assim por diante
- Bit válido-inválido anexado a cada entrada na tabela de páginas:
 - “válido” indica que a página associada está no espaço de endereço lógico do processo e, portanto, é uma página válida
 - “inválido” indica que a página não está no espaço de endereçamento lógico do processo
 - Ou usar page-table length register (PTLR)
- Quaisquer violações resultam em uma interrupção do tipo trap para o kernel

Bit válido (v) ou inválido (i) em uma tabela de página



Páginas Compartilhadas

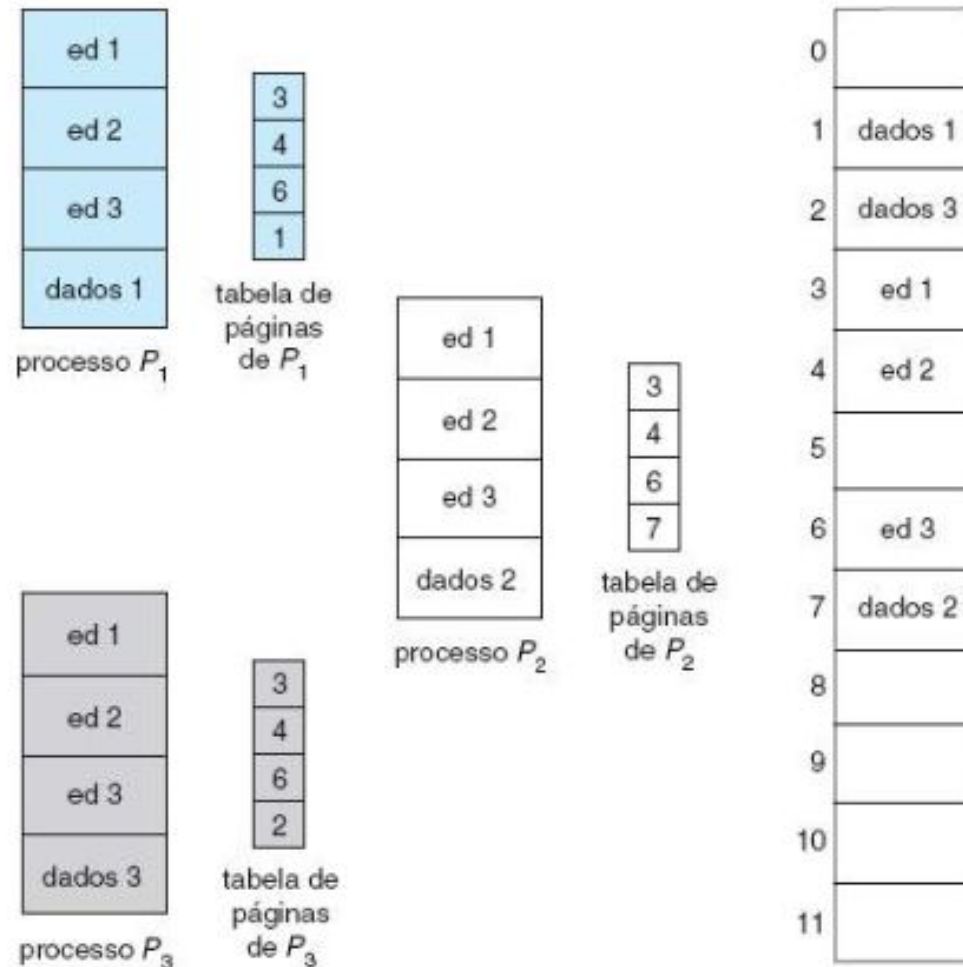
- Código compartilhado

- Uma cópia de código somente leitura compartilhada entre processos (ou seja, editores de texto, compiladores)
- Semelhante a vários threads compartilhando o mesmo espaço de processo
- Também útil para comunicação entre processos se o compartilhamento de páginas de leitura/gravação for permitido

- Código privado e dados

- Cada processo mantém uma cópia separada do código e dos dados
- As páginas para o código privado e os dados podem aparecer em qualquer lugar no espaço de endereço lógico

Exemplo de páginas compartilhadas



Estrutura da tabela de páginas

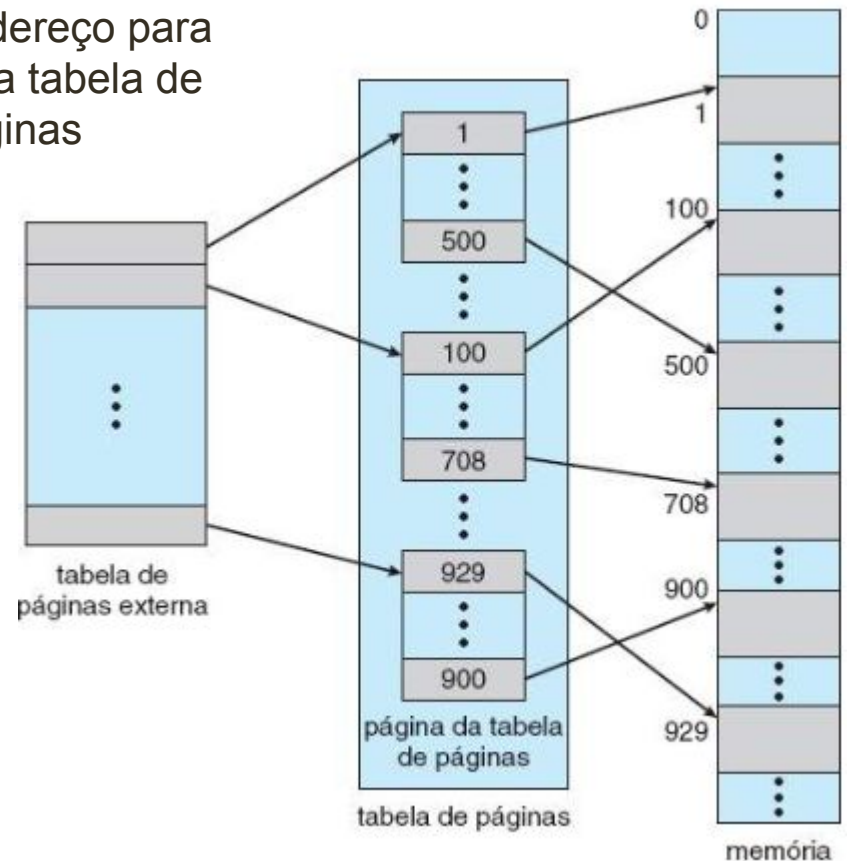
- As estruturas de memória para paginação podem ficar enormes usando métodos diretos
 - Considere um espaço de endereço lógico de 32 bits como em computadores modernos (2^{32})
 - Tamanho da página de 4 KB (2^{12})
 - A tabela de páginas teria $(2^{32}/2^{12}) = 2^{20}$ de entradas
 - Se cada entrada tiver 4 bytes $\rightarrow$ cada processo precisaria de 4 MB só para armazenar sua tabela de páginas
 - Ninguém reservar 4MB continuamente por processo na memória principal
 - A solução é dividir a tabela de páginas em unidades menores
 - **Paginação Hierárquica**
 - **Tabelas de página com hash**
 - **Tabelas de páginas invertidas**

- Soluções para dividir a tabela de páginas em unidades menores.
 - Paginação Hierárquica (Pesquisar)
 - Tabelas de página com hash (Pesquisar)
 - Tabelas de páginas invertidas (pesquisar)
 - Swapping

Tabelas de Páginas Hierárquicas

- Divida o espaço de endereço lógico em várias tabelas de páginas.
- Uma técnica simples é uma tabela de página de dois níveis
- Suporta endereços grandes (64bits)

Endereço para uma tabela de páginas



TLBs

Tabelas de Páginas Hierárquicas

Tabela de Páginas Externa (Nível 1): O endereço lógico aponta primeiro para esta tabela. Cada entrada nela não aponta diretamente para a memória física, mas sim para o endereço de uma página da tabela de páginas (segundo nível).

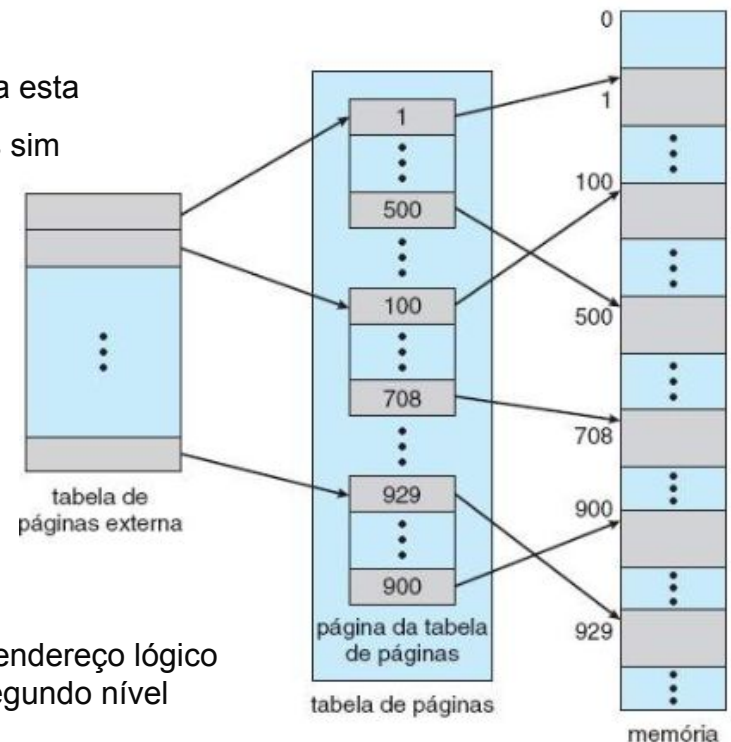
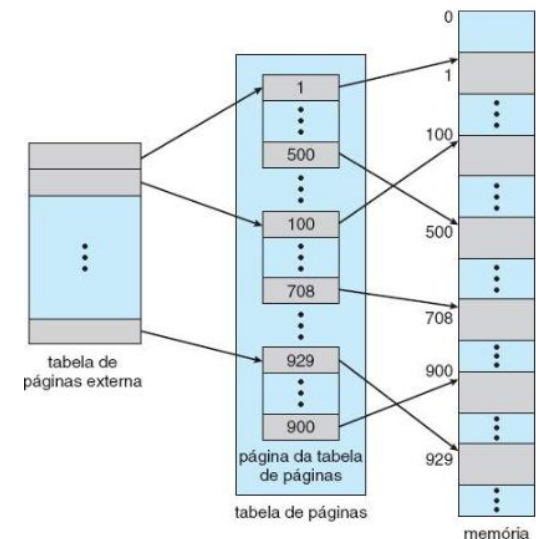


Tabela de Páginas (Nível 2): Esta tabela contém os mapeamentos reais. O endereço lógico fornece um índice que localiza a entrada específica dentro desta tabela de segundo nível

Memória Física (Frames): A entrada na tabela de nível 2 contém o número do quadro (frame) correspondente na memória RAM

Tabelas de Páginas Hierárquicas

- De modo geral essa estrutura é utilizada para economizar memória.
- Em vez de manter uma tabela de páginas única e gigante para todo o espaço de endereçamento (que ocuparia muita RAM mesmo para processos pequenos), o sistema só cria tabelas de segundo nível para as partes da memória que o processo realmente está utilizando.



Exemplo de Paginação em Dois Níveis

- Um endereço lógico (em uma máquina de 32 bits com tamanho de página de 1K) é dividido em:
 - um número de página consistindo de 22 bits
 - um deslocamento de página consistindo de 10 bits $\gg 2^{10}$
- Como a tabela de páginas é paginada, o número da página é dividido em:
 - um número de página de 12 bits
 - um deslocamento de página de 10 bits

Exemplo de Paginação em Dois Níveis

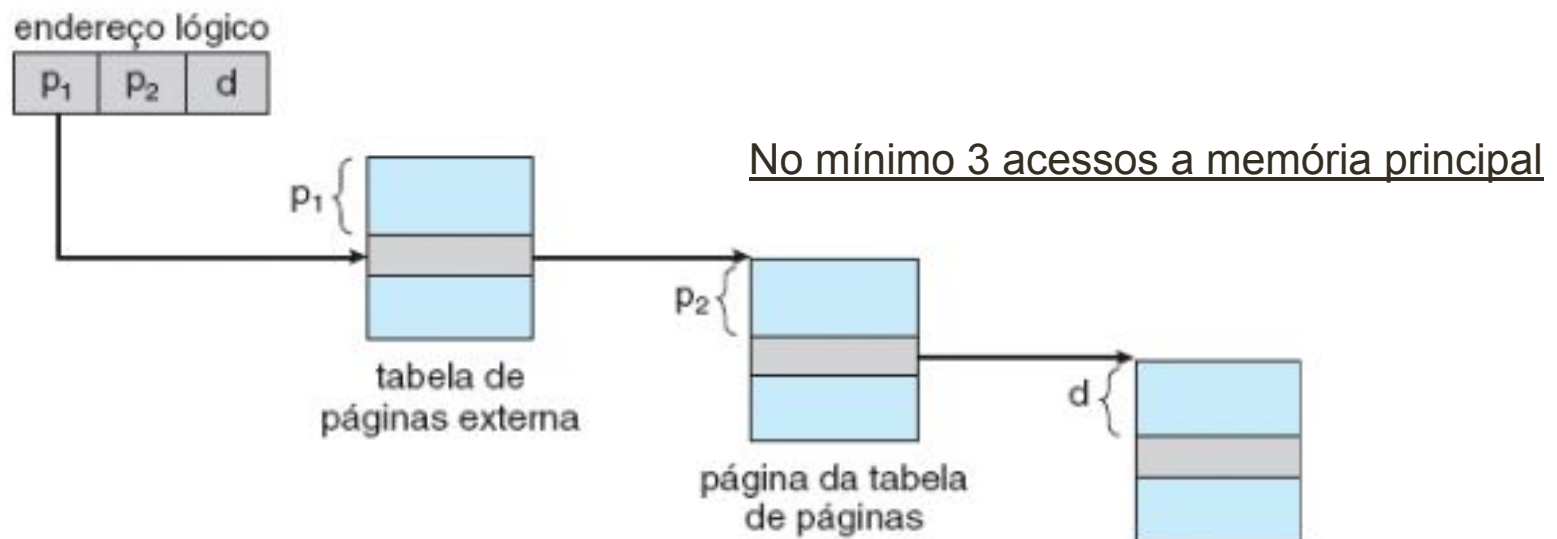
Endereço lógico é o seguinte:

- Onde p_1 é um índice na tabela de página externa e p_2 é o deslocamento dentro da página da tabela de página interna
 - Conhecida como tabela de página mapeada para frente

page number		page offset
p_1	p_2	d
12	10	10

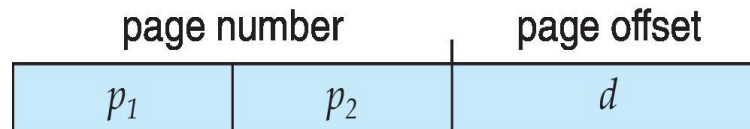
Conhecida como tabela de página mapeada para frente

1. O sistema usa P_1 para localizar uma entrada na tabela externa. Essa entrada aponta para a base de uma tabela de segundo nível.
2. Usa-se P_2 para encontrar a entrada específica nessa segunda tabela. Esta entrada contém o endereço da página física (frame) na memória RAM.
3. O deslocamento d é combinado com o endereço do frame para formar o endereço físico real.



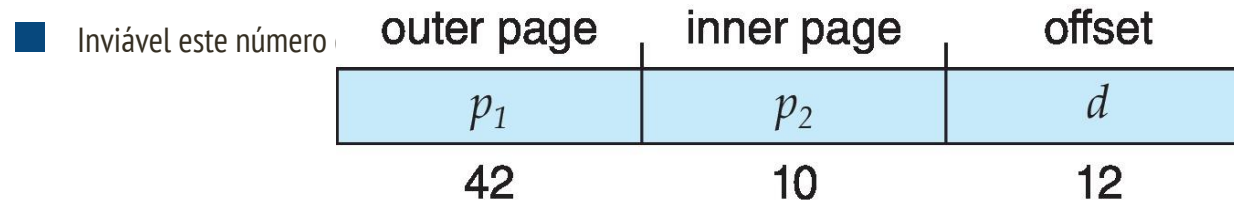
Espaço de endereço lógico de 64 bits

- O esquema de paginação de dois níveis não é suficiente
 - Um sistema com endereços lógicos de 64 bits pode teoricamente endereçar até 2^{64} bytes de memória.
 - Isso significa 16 exabytes de memória endereçável.
 - Mas a memória física real é muito menor – o desafio é gerenciar esse espaço lógico imenso com tabelas de páginas eficientes.



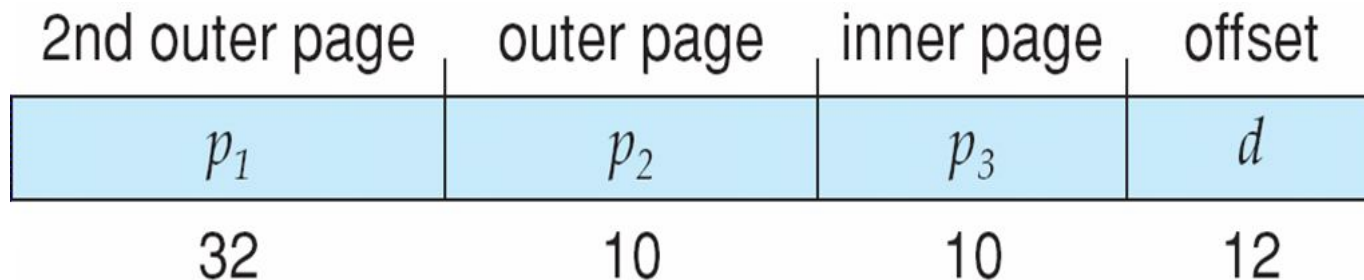
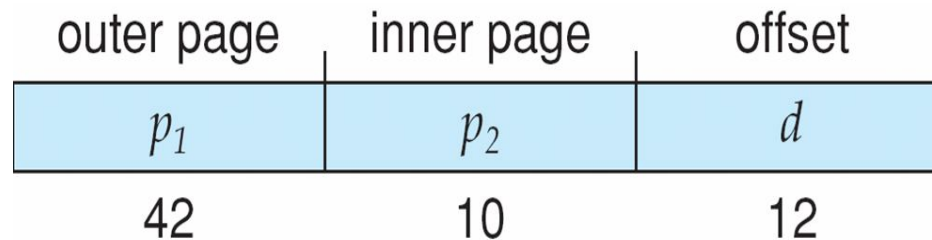
Espaço de endereço lógico de 64 bits

- O esquema de paginação de dois níveis não é suficiente
 - Se o tamanho da página for 4 KB (2^{12}), então para conseguir endereçar todos os bytes da pagina são necessários 12 bits. O restante ficam para o número de páginas. (52bits)
 - Então a tabela de páginas tem 2^{52} entradas (4PetaB)



- Usando esquema de dois níveis as tabelas de páginas internas podem ter 2^{10} entradas de 4 bytes
- Tabela de página externa tem 2^{42} entradas ou 2^{44} bytes (Tera)

Esquema de Paginação de Três Níveis



4GB

4 acessos à memória para chegar a um local de memória física

Tabelas de página com hash

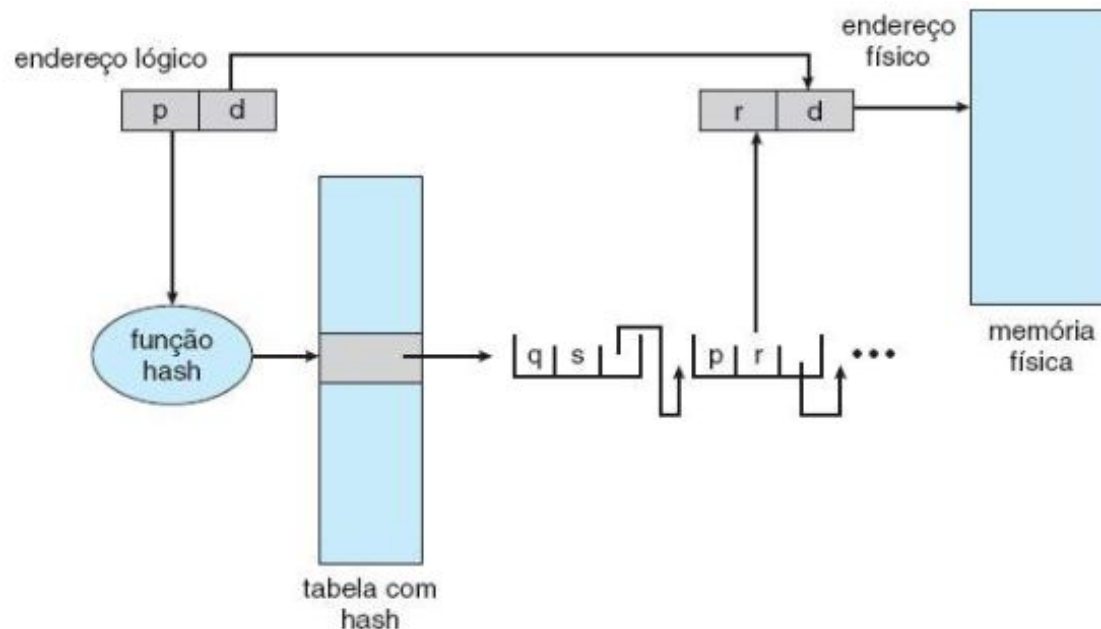
- Comum em espaços de endereços > 32 bits
- Preencher o máximo as possível de todas as entradas da tabelas de páginas.
- Aos invés de pegar o número da pagina lógica como indice, obten-se o número da pagina lógica a passa em uma função Hash.
- O número da página virtual é mapeado por hash na tabela de páginas.

Cada elemento contém:

- (1) o número da página virtual; (campo utilizado pela hash)
- (2) número de página física; (q)
- (3) um ponteiro para o próximo elemento. (indice da próxima linha da tabela)

Tabelas de página com hash

- É feito uma busca a partir do endereço gerado pela hash. Números de páginas virtuais são comparados em busca de uma correspondência (frame físico). É feito uma busca a partir do endereço gerado pela hash.
- O número da página virtual é igual ao número da página virtual do endereço lógico, senão próximo.



Hash

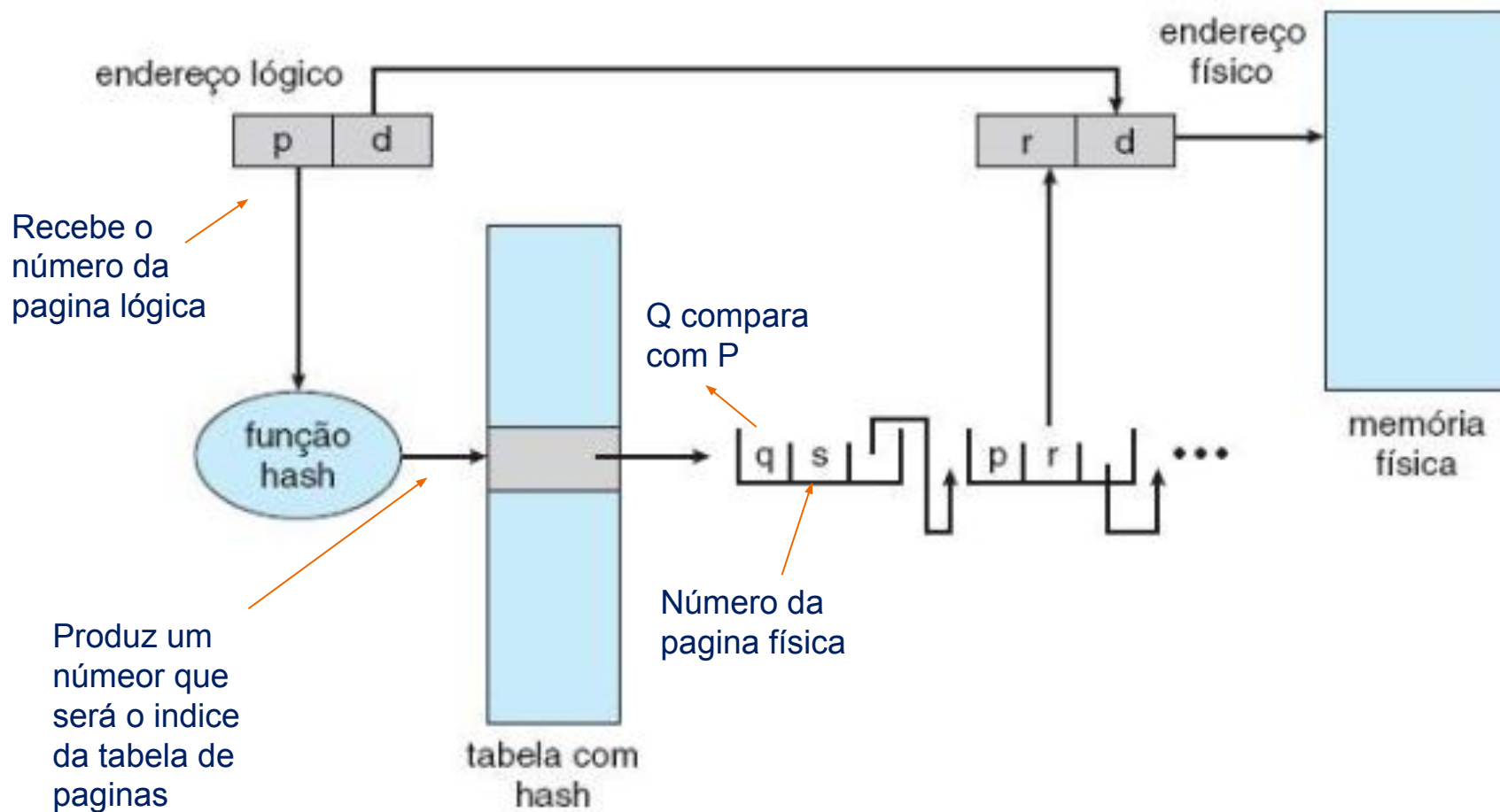


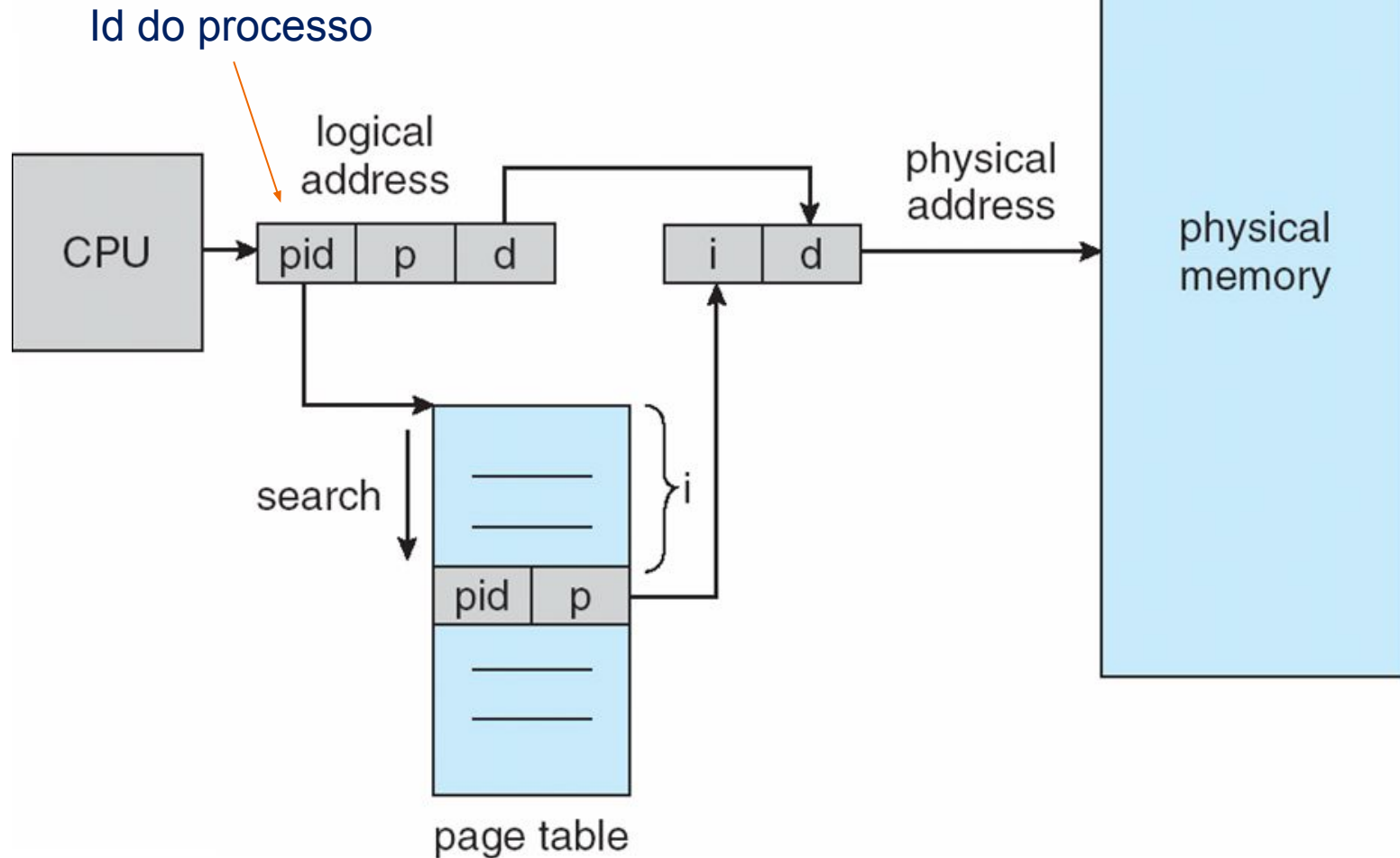
Tabela de página invertida

- Pesquise sobre tabela de página invertida
 - Compare com os modelos analisados anteriormente
 - Pontos positivos e negativos
 - Melhor caso de uso

Tabela de página invertida

- Em vez de cada processo ter uma tabela de páginas e rastrear todas as páginas lógicas possíveis, rastreie todas as páginas físicas
- Uma entrada para cada página real da memória
- A entrada consiste no endereço virtual da página armazenada naquele local de memória real, com informações sobre o processo que possui essa página
- Diminui a memória necessária para armazenar cada tabela de página, mas aumenta o tempo necessário para pesquisar a tabela quando ocorre uma referência de página
- Use a tabela de hash para limitar a pesquisa a uma — ou no máximo algumas — entradas da tabela de páginas
 - TLB pode acelerar o acesso
- Mas como implementar a memória compartilhada?

Arquitetura de tabela de página invertida



Swapping

Swapping

- Definição
- Conceito
- Visão esquemática de Swapping
- Tempo de troca de contexto, incluindo Swapping
- Swapping em sistemas móveis
- Swapping com paginação

Swapping

- Um processo pode ser trocado temporariamente da memória para uma memória de retaguarda, em seguida, trazido de volta à memória para execução contínua
 - O espaço total da memória física dos processos pode exceder a memória física
- Armazenamento de apoio – disco rápido grande o suficiente para acomodar cópias de todas as imagens de memória para todos os usuários; deve fornecer acesso direto a essas imagens de memória
- Roll out, roll in – variante de troca usada para algoritmos de escalonamento baseados em prioridade; o processo de prioridade mais baixa é trocado para que o processo de prioridade mais alta possa ser carregado e executado.

Swapping

- A maior parte do tempo de troca é o tempo de transferência; o tempo total de transferência é diretamente proporcional à quantidade de memória trocada.
- O sistema mantém uma fila pronta de processos prontos para execução que possuem imagens de memória em disco.

Swapping

- O retorno deve ser para o mesmo endereço físico?
 - Depende do modelo de ligação/linkagem de endereço
 - Deve-se considerar E/S pendentes para o espaço de memória do processo
- A permuta torna possível que o espaço de endereçamento físico, de todos os processos exceda a memória física real do sistema, aumentando assim o grau de multiprogramação no sistema

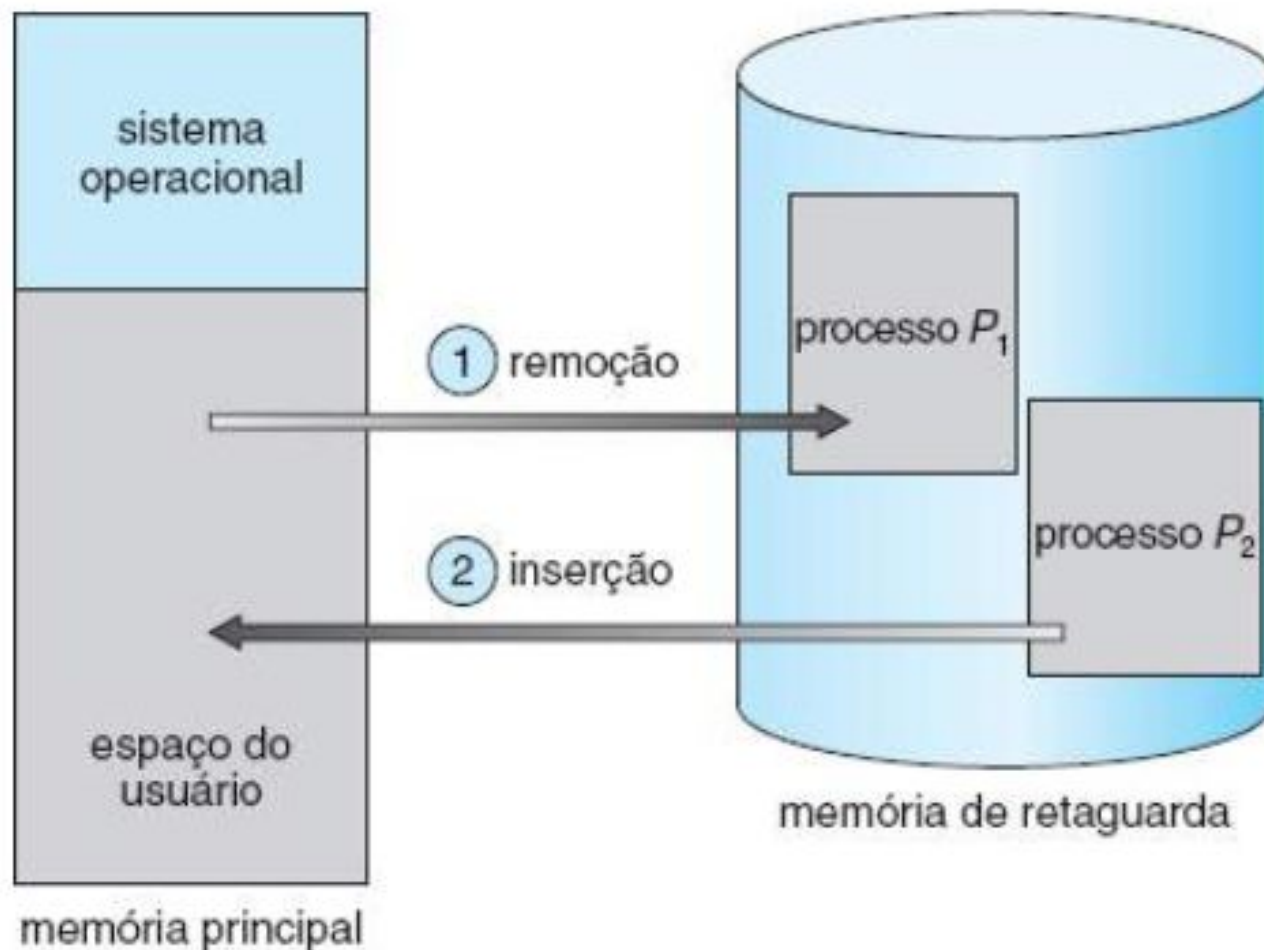
Swapping

- Versões modificadas de troca são encontradas em muitos sistemas(ex: UNIX, Linux e Windows)
 - Swapping normalmente desativado
 - Iniciado se mais do que a quantidade limite de memória alocada
 - Desativado novamente quando a demanda de memória for reduzida abaixo do limite

Swapping - conceito

- Estabelece uma cópia de toda a memória para todos os usuários;
- O sistema mantém uma fila de prontos composta por processos cujas imagens da memória estão na MR ou na MP e prontos.
- Scheduler; executa o processo pronto
- Dispatch; verifica se as informações estão em memória
 - Remove um processo correntemente;

Visão esquemática do Swapping



Tempo de troca de contexto, incluindo Swapping

- Se os próximos processos a serem colocados na CPU não estiverem na memória, é necessário trocar um processo e trocar no processo de destino
- O tempo de troca de contexto pode ser muito alto
- Troca de processo de 100 MB para disco rígido com taxa de transferência de 50 MB/s
 - Tempo de troca de 2000 ms
 - Mais troca de processo do mesmo tamanho
 - Tempo total do componente de troca de troca de contexto de 4000ms (4 segundos)
- Pode reduzir se reduzir o tamanho da memória trocada – sabendo quanta memória realmente está sendo usada
 - Chamadas do sistema para informar o SO sobre o uso da memória via `request_memory()` e `release_memory()`

Tempo de troca de contexto, incluindo Swapping

- Tempo Alto
 - Processo do usuário tenha um tamanho de 100 MB
 - Memória Secundária (disco) taxa de transferência de 50 MB por segundo
- $100 \text{ MB} / 50 \text{ MB por segundo} = 2000\text{ms}$ para cada operação
- Tempo de transferência = permuta
- Memória completa não disponível
 - Diminui a perda com swapping nas parte mais importantes dos processos.

Tempo de troca de contexto, incluindo Swapping

- Portanto, um processo com requisitos de memória dinâmica (reduzir o tamanho da memória trocada) terá de emitir chamadas de sistema [request_memory () e release_memory ()] para informar ao sistema operacional as mudanças em suas necessidades de memória.
 - Devem ser feitas apenas quando:
 - Não há processo com I/O pendente
 - Executar operações de I/O somente em buffers do sistema operacional

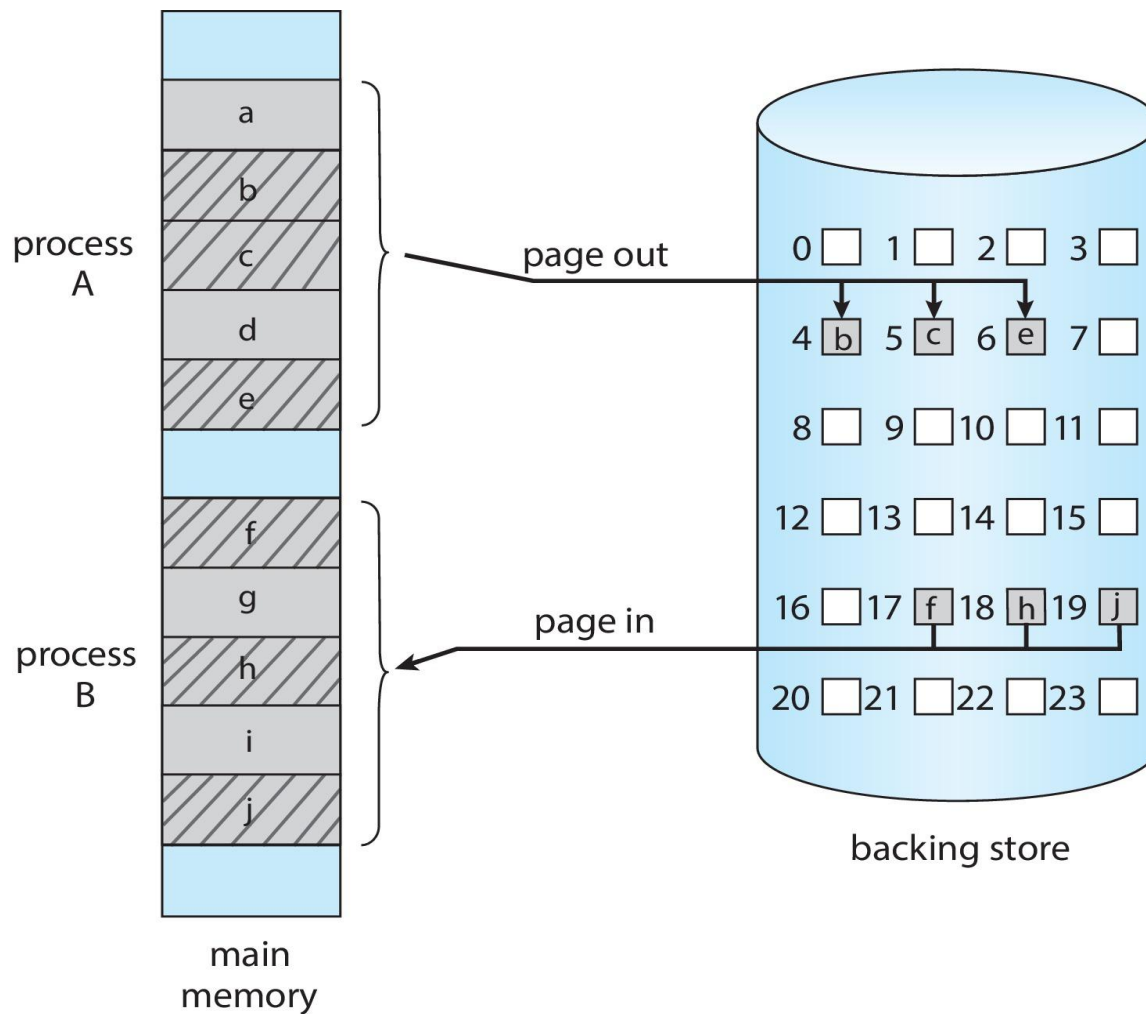
Swapping em sistemas móveis

- Normalmente não suportado
 - baseado em memória flash
 - Pequena quantidade de espaço
 - Número limitado de ciclos de gravação
 - Baixa taxa de transferência entre a memória flash e a CPU na plataforma móvel
- Em vez disso, use outros métodos para liberar memória se estiver baixa
 - O iOS pede que os aplicativos abandonem voluntariamente a memória alocada
 - Dados somente leitura descartados e recarregados do flash, se necessário
 - A falha na liberação pode resultar em rescisão

Swapping com paginação

- Combinação das técnicas de gerenciamento de memória: a divisão dos processos em páginas (paginação) e a movimentação dessas páginas entre a memória principal e a memória secundária (swapping).
- Quando a memória RAM está cheia, páginas menos utilizadas são transferidas para o disco (page out) e, quando necessário, são trazidas de volta (page in).
- A memória secundária utilizada nesse processo é chamada de *backing store*.
 - Essa abordagem permite um uso mais eficiente da memória física e melhora a capacidade do sistema de executar múltiplos processos

Swapping com paginação



Exercícios sobre Gerência de memória

Rodar o código no github Memory Cost:

No linux :

main.cpp
stdafx.h
stdafx.cpp
targetver.h



! (Visual Studio / Windows)
! (Visual Studio / Windows)
! (Windows SDK)

Conteúdo:

- main()
- lógica do benchmark
- desempenho
- alocação (new)
- acesso à memória (memset, leitura)

Código Github - Memory Cost

Rodar o código no github e responder às seguintes questões:

1. Qual o tempo estimado para alocar 32MB na RAM?
2. Se rodar novamente este tempo será o mesmo?
Este é um tempo real ou virtual? Por que?
3. Qual seção da pilha o gerenciador de memória atua neste ponto?
4. Qual o tempo estimado para escrever na memória?
5. Qual o tempo estimado para leitura na memória?
6. Quais fatores impactam na leitura para que seja mais rápida que a escrita ?

Código Github - Memory Cost

7. Qual o tempo total da manipulação?
8. Quanto tempo para desalocar a memória e atualizar a estrutura da heap?
9. A variavel SUM possui qual valor , por que?
10. Este valor é sempre o mesmo?
11. Por que mesmo sem escrever (desalocando) temos custo?
12. Qual a conclusão sobre Alocação de memoria e Acesso, dentro deste cenário?
13. Crie uma tabela com as principais operações/chamadas no código e faça uma relação custo e motivo.

Código Github - Memory Cost

Operação	Custo	Motivo
New		
Delete		
Write		
Read		
Alloc + write		
Alloc + read		

14. Altere o tamanho do buffer de 32MB para 64MB, 128MB e 256 MB faça uma tabela comparativa dos tempos de escrita e leitura.

Tamanho (MB)	Write (s)	Read (s)
32	0.1009	0.0855
64	?	?
128	?	?
256	?	?

Código Github

Rodar o código no git hub e reponder as seguintes questões:

Busy waiting to raise the CPU frequency...

0.0000 s to allocate 32 MB 100 times.

0.0000 s to allocate 32 MB 100 times (0.0000 s to delete).

0.1009 s to write 32 MB 100 times.

0.0855 s to read 32 MB 100 times, sum = 838860800.

0.4334 s to allocate and write 32 MB 100 times (0.0296 s to delete).

0.1944 s to allocate and read 32 MB 100 times, sum = 0.



UNIVALI

Universidade do Vale do Itajaí
Escola Politécnica

Fim



UNIVALI

Universidade do Vale do Itajaí
Escola Politécnica

Memória Principal - Sistemas Operacionais

Prof. Michael Douglas C Alves

Gerência de Memória

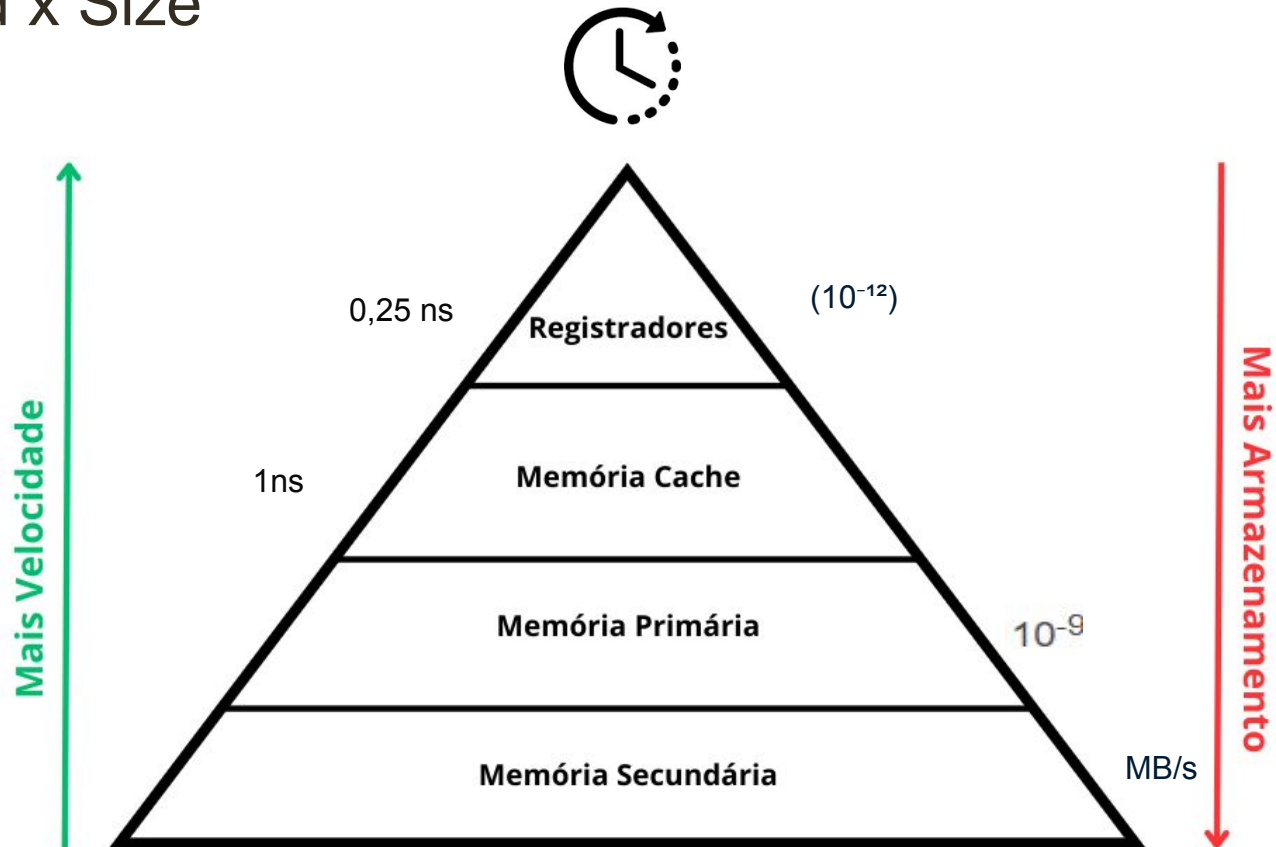
- Conceitos
- Segmentação
- Alocação de memória contínua
Como que é gerenciada
- Paginação
Estrutura da tabela de páginas
- Swapping*

Gerenciamento

- Como a memória principal é gerenciada pelo SO
 - Monoprogramada – um único programa
 - Multiprogramada – divisão da memória em várias partes
- Melhor cenário
 - Memória volátil
 - Velocidade
 - Capacidade de armazenamento
 - Fornecer um pedaço da memória para cada processo.

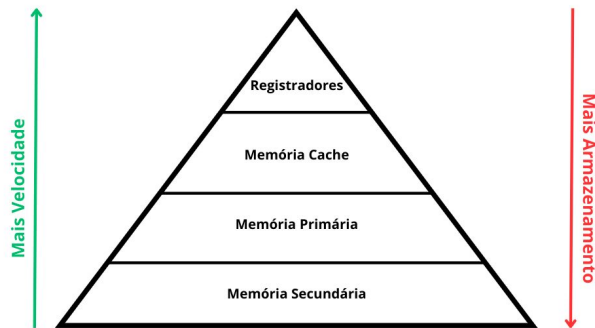
Hierarquia de Memória

- Speed x Size



Hierarquia de Memória

- Speed x Size



Registradores: diretamente dentro da CPU (Processador) do computador e em quantidade muito pequena.

Memória Cache: armazenamento das informações que são utilizadas com frequência pelo processador.

Memória primária: (RAM).

Memória secundária: os elementos de armazenamento físico

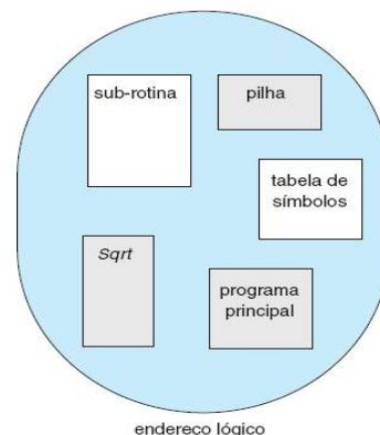
Gerenciamento

- Gerenciador de memória
 - Manter o controle das partes utilizadas e as não utilizadas (Médio prazo)
 - Garantia de recursos aos processos
 - Admitindo ou encerrando
 - Cada processo deve acessar apenas seu espaço de memória.
 - Registradores de base e limite para definir o intervalo válido para acessar a memória
 - deve-se alocar a memória de forma eficiente ao maior número de processos

- O programa deve ser trazido (do disco) para a memória e deve ser transformado em um processo para que ele seja executado
 - A memória principal e os registradores são apenas o armazenamento, que a CPU pode acessar diretamente.
 - Somente o SO pode configurar os registradores (instruções privilegiadas)
- A unidade de memória só vê um fluxo de:
endereços + solicitações de leitura, ou endereço + dados e solicitações de gravação
- O acesso de registrador é feito em um clock da CPU (ou menos)
A memória principal pode levar muitos ciclos, causando uma **Stall** (CPU solicita dados da RAM, ela precisa esperar vários ciclos de clock)
- O cache fica entre a memória principal e os registradores da CPU
 - Proteção da memória necessária para garantir o funcionamento correto (localidade e referência)

Segmentação

- É um esquema de gerenciamento de memória que sustenta a visão do programador.
- A segmentação oferece um mecanismo de memória que mapeia a visão do programador para a memória física real.
- O sistema dispõe de maior liberdade para gerenciar a memória, enquanto o programador usufrui de um ambiente de programação mais produtivo.

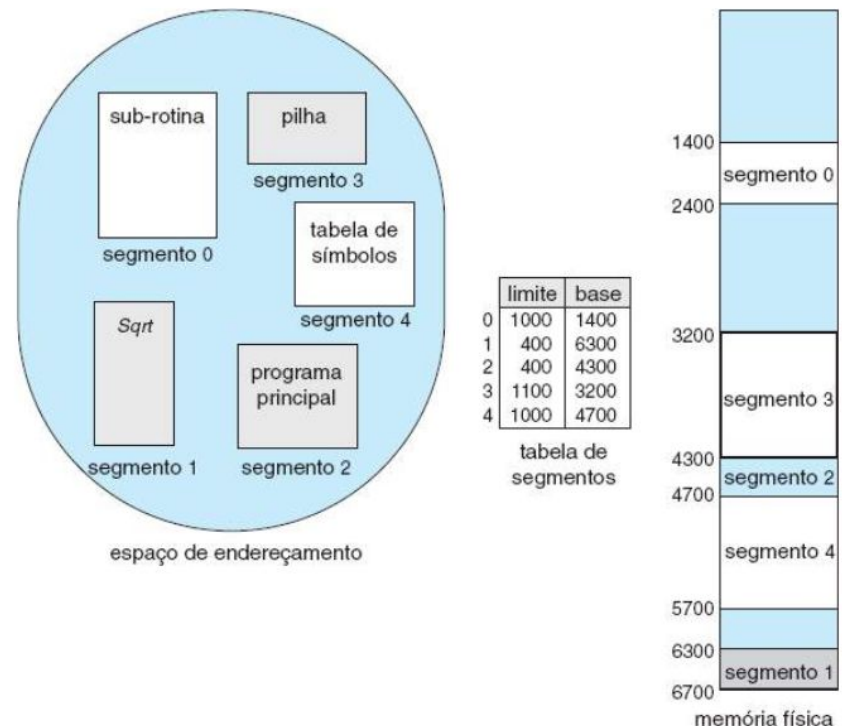


Segmentação

- A alocação vai depender da lógica do programa.
- O mapeamento é feito através das tabelas de mapeamento de segmentos

Normalmente, o compilador constrói automaticamente segmentos que refletem o programa de entrada. Um compilador C pode criar segmentos separados para os elementos a seguir:

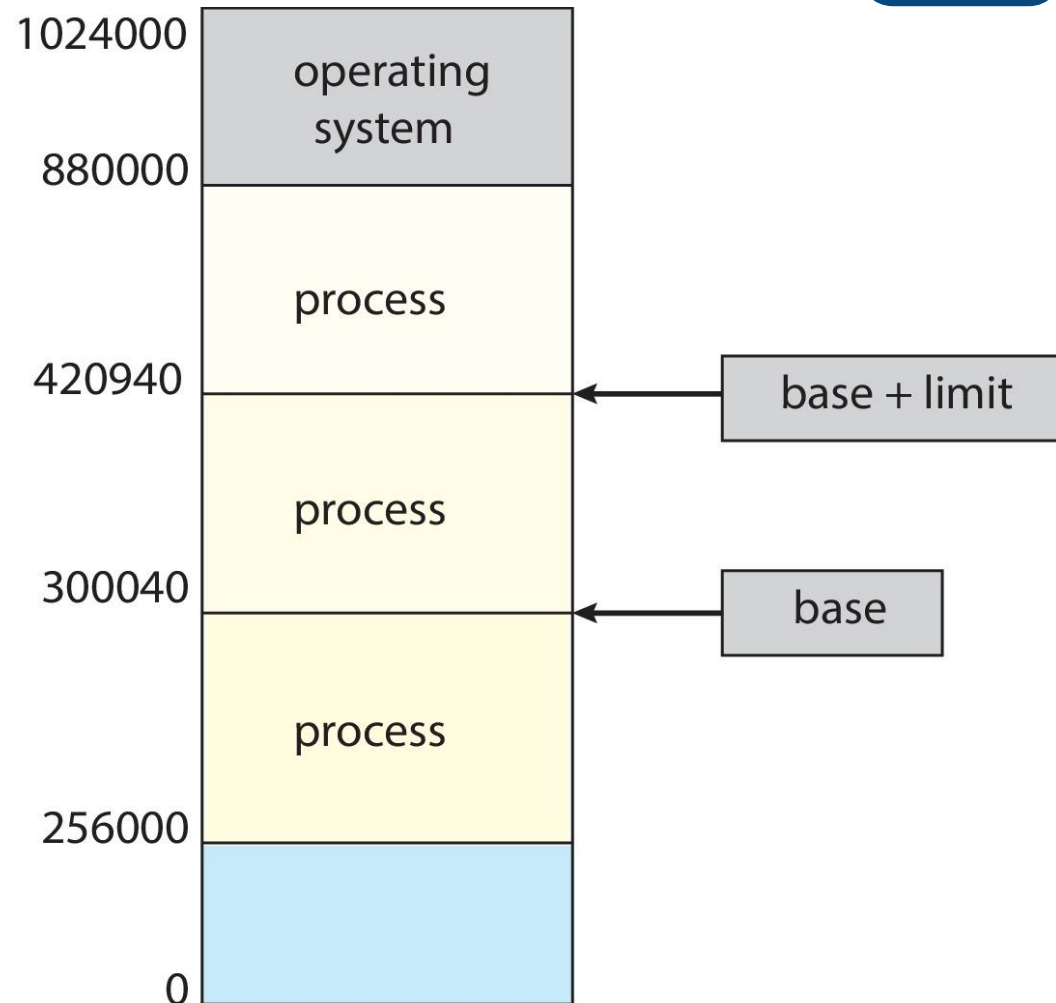
- 1.O código
- 2.Variáveis globais
- 3.O heap a partir do qual a memória é alocada
- 4.As pilhas usadas para cada thread
- 5.A biblioteca C padrão.



Proteção- realocação dinâmica

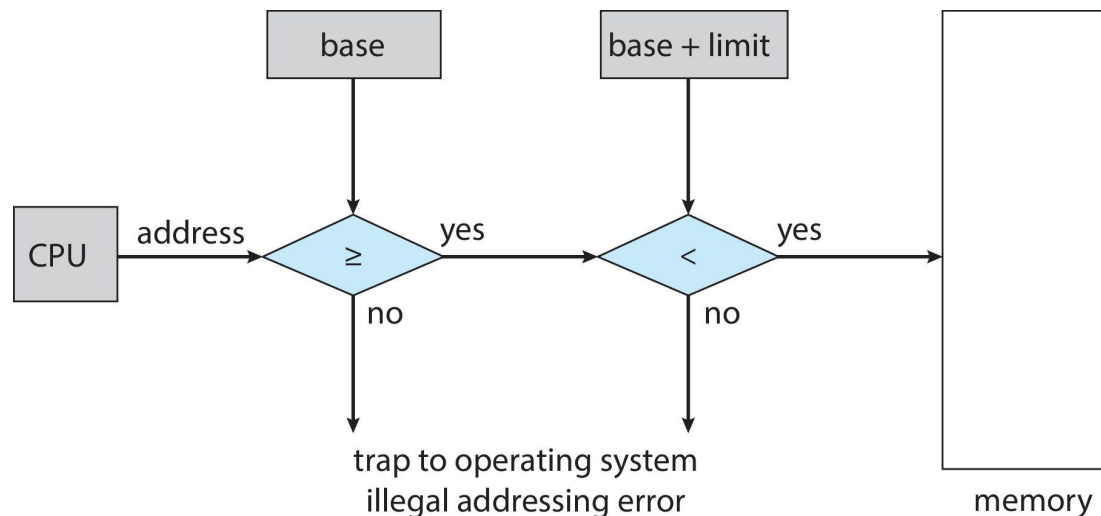
- Necessidade de proteger para que um processo só possa acessar os endereços em seu espaço de endereço

Podemos fornecer essa proteção usando um par de registradores de base e limite para definir o espaço de endereço lógico de um processo



Proteção de endereço de hardware

- A CPU deve verificar cada acesso à memória gerado no modo de usuário, para ter certeza de que está entre base e limite para esse usuário



- As instruções para carregar os registradores de base e limite são privilegiadas (SO-Assembly)

Linkagem de endereço

- Programas em disco aguardam na fila de entrada para serem carregados na memória.
 - Sem configuração, carregam no endereço 0000.
 - Não é ideal que processos de usuário carreguem no endereço físico 0000.
 - **Endereços se transformam:**
 - Pelo programador temos o código-fonte: simbólicos (cout).
 - Código compilado: endereços relocáveis (ex: "14 bytes do início do módulo").
 - **Linker/loader:** Vincula os endereços relocáveis a endereços absolutos/físicos (ex: transforma uma posição relativa em um endereço real como 74000→74014).
 - Endereços são traduzidos de simbólicos → relocáveis → físicos
- Momento de compilação, carregamento e execução.

Linkagem - Binding

- Processo de associar instruções e dados aos seus endereços reais na memória
 - A vinculação de instruções e dados a endereços de memória pode ocorrer em três estágios diferentes:
 - 1- Tempo de compilação: Ocorre quando o compilador sabe exatamente em que endereço o programa residirá na memória. Se o local mudar futuramente, o programa precisará ser recompilado.
 - Para mudar o programa de lugar, era necessário alterar o código-fonte e recompilar tudo para o novo endereço.

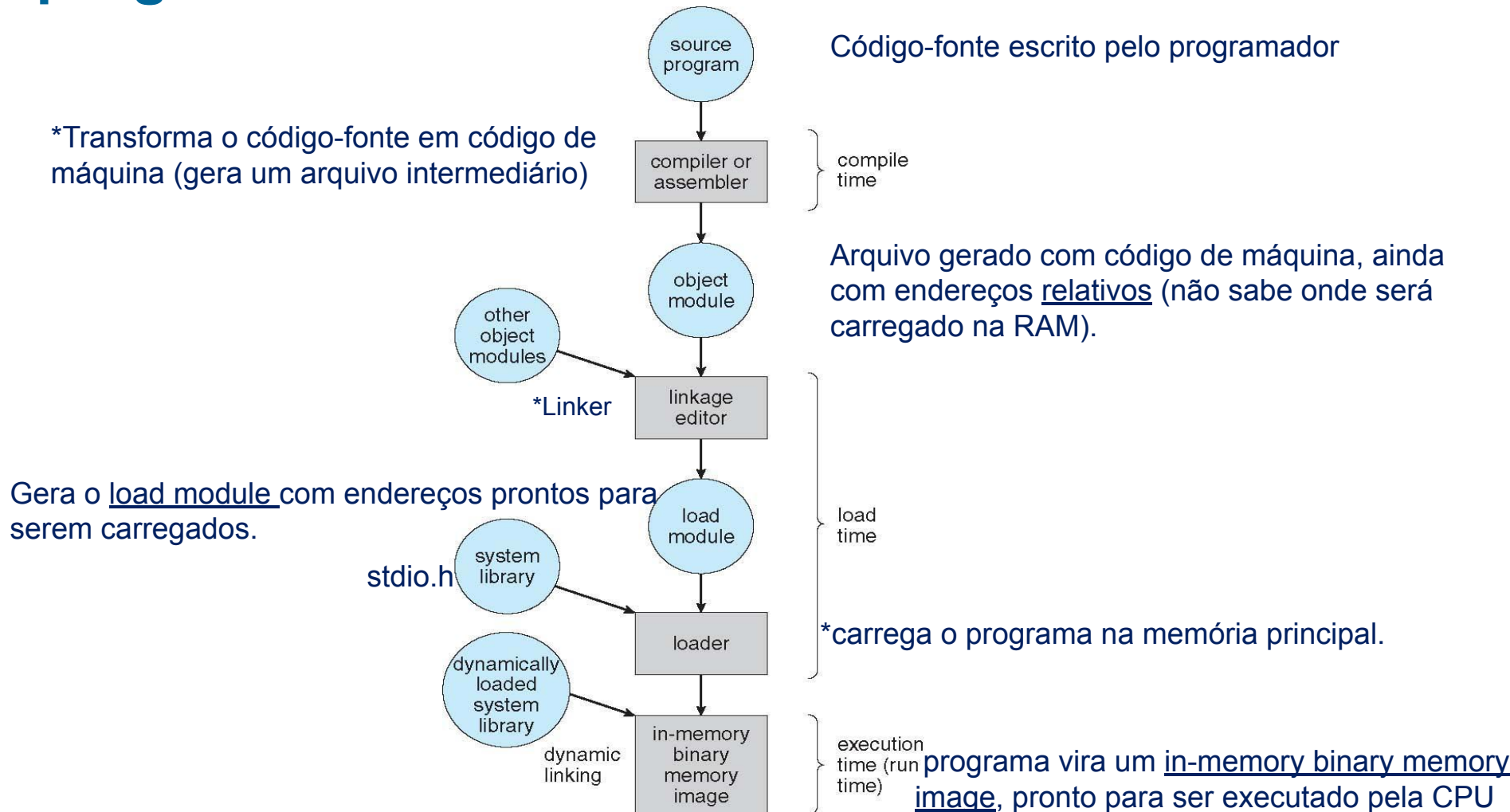
Linkagem - Binding

- Processo de associar instruções e dados aos seus endereços reais na memória
 - A vinculação de instruções e dados a endereços de memória pode ocorrer em três estágios diferentes:
 - 2- Tempo de carregamento: Caso o endereço não seja conhecido, o compilador tem que gerar um código relocável
 - Funcionamento das DLLs (Dynamic Link Libraries) no Windows, ou arquivos .so (Shared Objects) no Linux
 - O Loader (carregador) do Windows lê o `.exe` e vê que ele precisa da `math.dll`.
 - O Windows carrega a `math.dll`.

Linkagem - Binding

- Processo de associar instruções e dados aos seus endereços reais na memória
 - A vinculação de instruções e dados a endereços de memória pode ocorrer em três estágios diferentes:
 - 3- Tempo de execução: o processo pode ser movido durante sua execução de um segmento de memória para outro, ou seja o processo muda deposição na memoria durante a própria execução. (MMU)
 - Precisa de suporte de hardware para mapas de endereços (por exemplo, registradores de base e limite)

Processamento em várias etapas de um programa de usuário

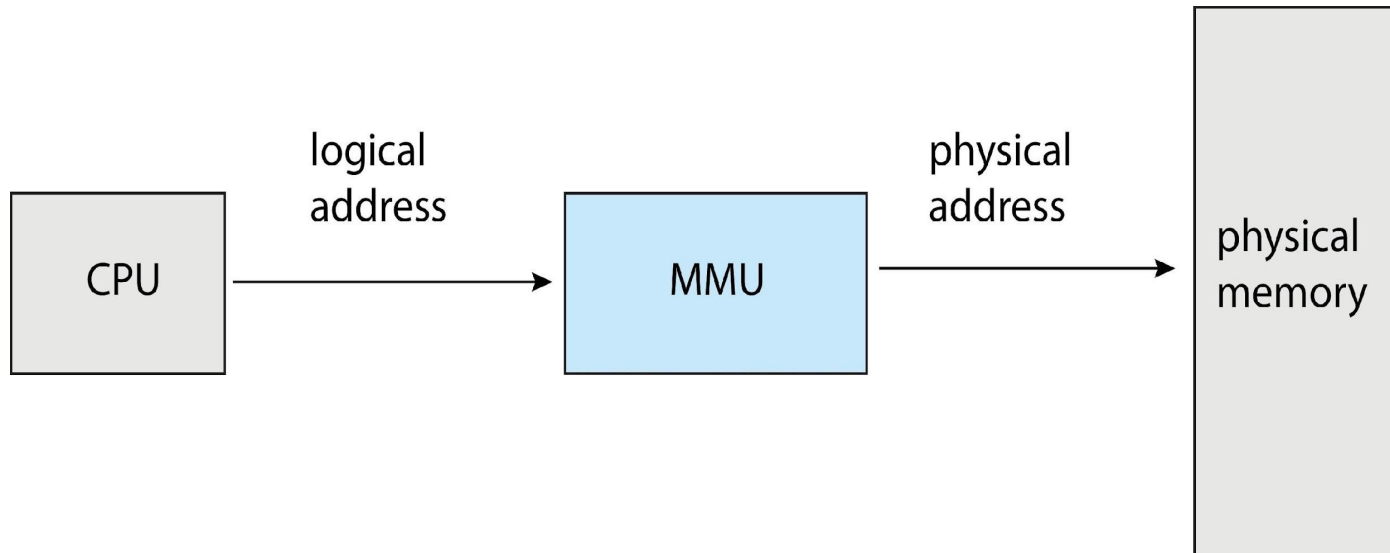


Espaço de endereço lógico vs. físico

- O conceito de separação entre espaço de endereçamento lógico e físico é fundamental para o gerenciamento adequado da memória.
 - Endereço lógico – gerado pela CPU; também conhecido como endereço virtual
 - Endereço físico – endereço visto pela unidade de memória
 - Os endereços lógicos e físicos são os mesmos em esquemas de vinculação de endereço em **tempo de compilação** e **tempo de carregamento**;
 - Os endereços lógicos (virtuais) e físicos diferem no esquema de vinculação de endereço em **tempo de execução**
- **Espaço de endereço lógico**: é o conjunto de todos os endereços lógicos gerados por um programa.
Espaço de endereço físico é o conjunto de todos os endereços físicos gerados por um programa.

Memory-Management Unit (MMU)

- Dispositivo ou componente de hardware que, em tempo de execução, mapeia o endereço virtual para o físico.



Memory-Management Unit (MMU)

- Considere um esquema simples: uma generalização do esquema de registrador de base.

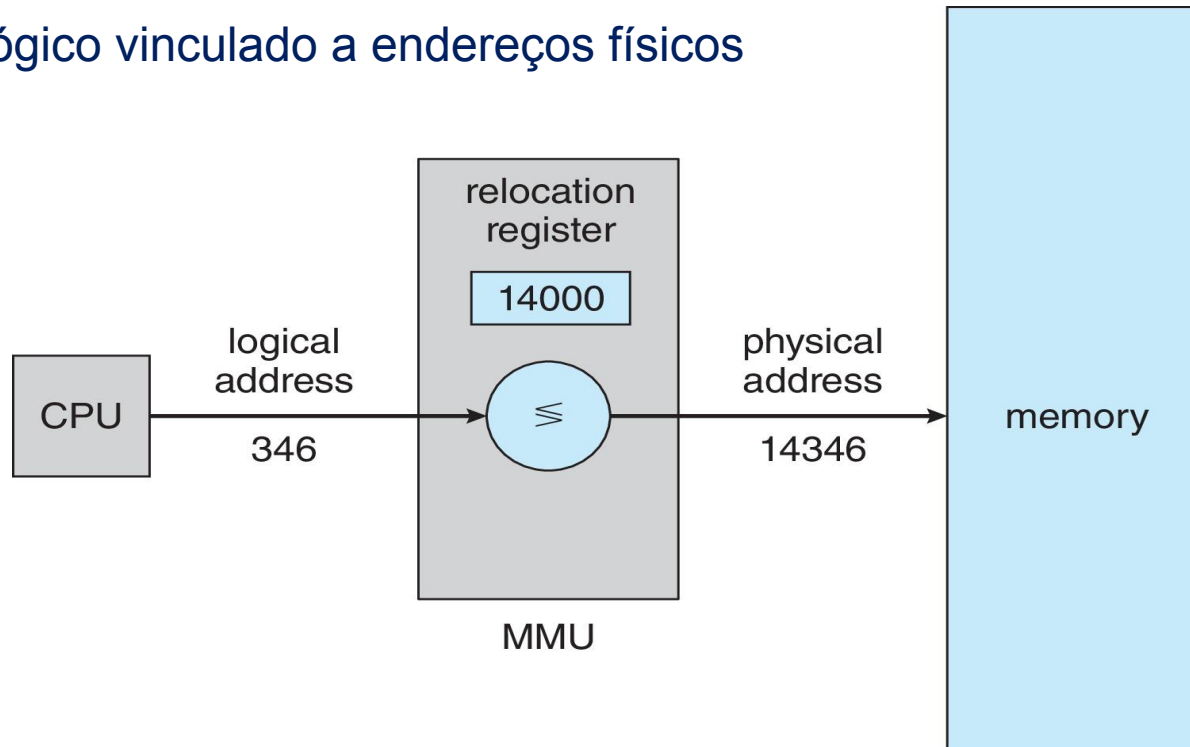
O registrador base agora chamado de registrador de *offset*.

O valor do registrador de offset é adicionado a todo endereço gerado por um processo de usuário no momento em que é enviado para a memória.

O programa de usuário lida com endereços lógicos; ele nunca vê os endereços físicos reais.

Memory-Management Unit (MMU)

Endereço lógico vinculado a endereços físicos



A linkagem em tempo de execução ocorre quando é feita referência ao local na memória.

Carregamento/Carga dinâmico(a)

- Técnica que permite carregar partes do processo do disco para a memória dinamicamente.
- A rotina não é carregada até que seja chamada
 - Melhor utilização do espaço de memória
- Todas as rotinas mantidas em disco em formato de carga realocável:
 - Útil quando grandes quantidades de código são necessárias para lidar com casos que ocorrem com pouca frequência
- Nenhum suporte especial do sistema operacional é necessário
 - Implementado através do design do programa: ex `dlopen()` e `dlclose()` (carrega e descarrega lib)
O sistema operacional pode ajudar fornecendo libs para implementar o carregamento dinâmico

Métodos de Vinculação estática x dinâmica de end. na memória

- Vinculação estática – Código de bibliotecas externas são incorporados diretamente em arquivo executável. (Imagem do programa).
 - O programa carrega todas as bibliotecas necessárias durante a compilação, resultando num executável independente.
 - Programa não depende de bibliotecas externas serem carregadas durante a execução.
- Vinculação dinâmica – durante a execução.
- Fragmento de código no SO: stub, é usado para localizar a rotina apropriada da biblioteca. Chama o loader do SO para buscar a função na biblioteca compartilhada (.so ou .dll).
- O Stub substitui a si mesmo como endereço da rotina e a executa. O SO verifica se a rotina está no endereço de memória dos processos;
 - Se não estiver no espaço de endereço, adicione ao espaço de endereço

Métodos de Ligação dinâmica

- A vinculação dinâmica é particularmente útil para bibliotecas.
 - Sistema também conhecido como bibliotecas compartilhadas: várias aplicações compartilhem da mesma
 - Considere a aplicabilidade para corrigir bibliotecas do sistema : bibliotecas vinculadas dinamicamente podem ser atualizadas sem que os aplicativos que as usam precisem ser recompilado.
 - Bibliotecas de terceiros
 - Pode não ser possível.

Alocação contígua de memória

- Recurso limitado, deve alocar de forma eficiente.

A alocação contígua é um dos primeiros métodos:

- Cada processo ocupa uma área contínua da memória (sem divisões externas).

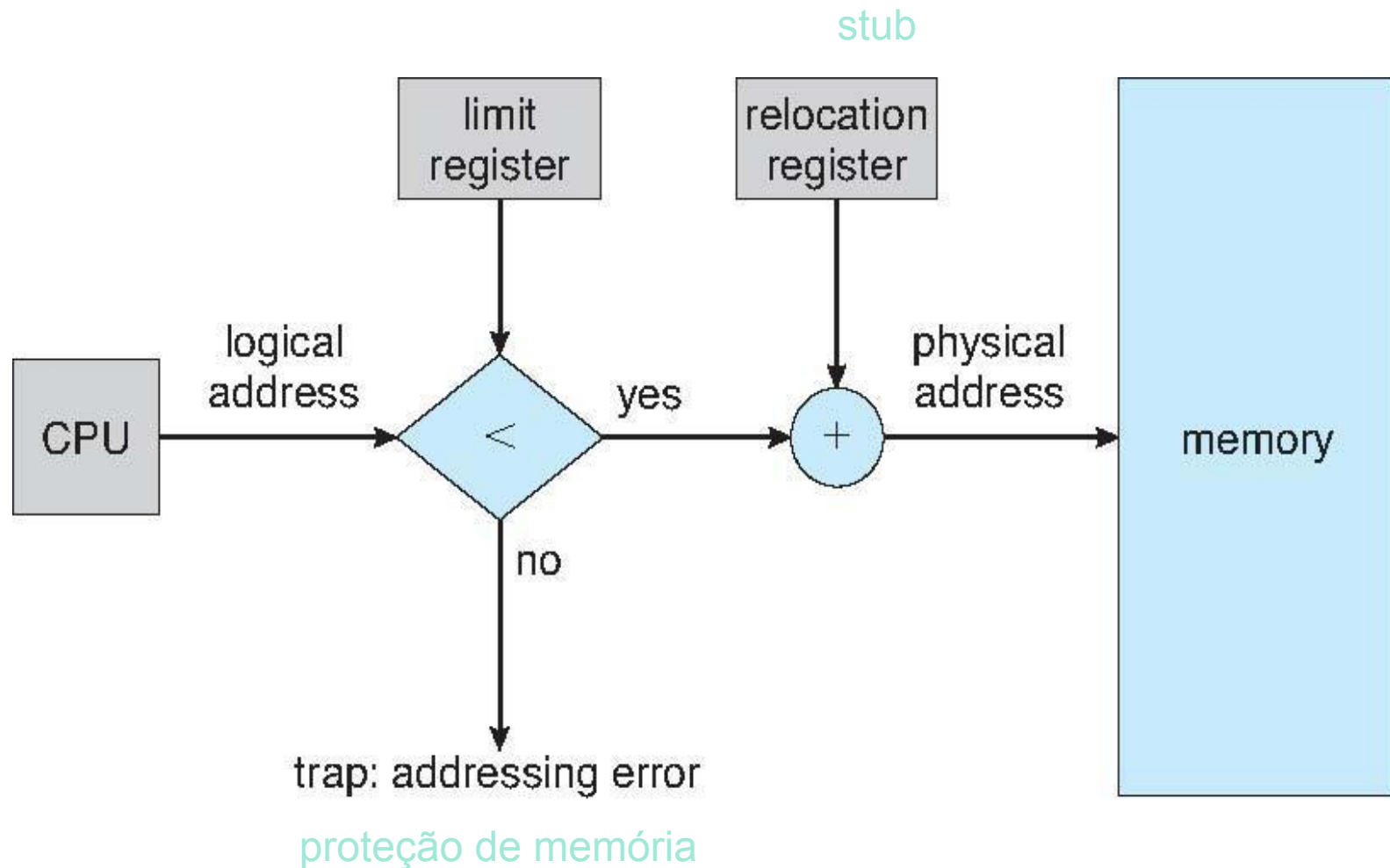
A organização da memória é dividida em duas partições:

- Parte baixa: Reservada para o Sistema Operacional (SO) e o vetor de interrupções, geralmente ocupando endereços de memória menores.
- Parte alta: Destinada aos processos do usuário, localizados em endereços de memória superiores (Programas)

Alocação contígua de memória

- Registradores de realocação usados para proteger os processos do usuário uns dos outros e contra a alteração do código e dos dados do sistema operacional:
 - O registrador base: Armazena o endereço físico inicial (o menor) alocado para o processo.
 - O registrador de limite: Define o tamanho do segmento de memória. Qualquer endereço lógico gerado pelo processo deve ser menor que esse valor para ser considerado válido.
 - MMU mapeia o endereço lógico dinamicamente
 - Pode então permitir ações como o código do kernel sendo transitório e o kernel mudando de tamanho.
 - Kernel não precisa ficar engessado.
 - Sistema operacional ganha flexibilidade para mover, aumentar ou diminuir o kernel na memória.

Suporte de hardware para realocação e registrador de limite



Partição variável

- Alocação de várias partições

- Grau de multiprogramação limitado pelo número de partições;

Tamanhos de partição variável para eficiência (dimensionados de acordo com as necessidades de um determinado processo);

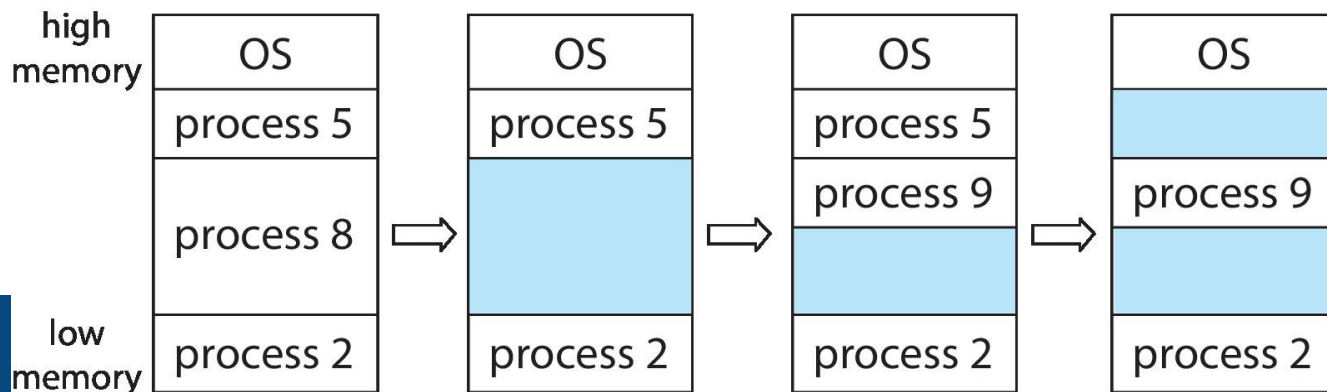
Buraco – bloco de memória disponível; buracos de vários tamanhos estão espalhados por toda a memória;

Quando um processo chega, ele recebe memória de um parte grande o suficiente para acomodá-lo

Processo de saída libera sua partição e partições livres adjacentes combinada;

O sistema operacional mantém informações sobre:

a) partições alocadas b) partições livres (buracos)



Problema de armazenamento com alocação dinâmica

- Como satisfazer um pedido de tamanho n de uma lista de buracos (memória) livres?
 - First-fit: Aloque a primeira lacuna disponível e tamanho suficiente (deixa pequenos buracos)
 - Best-fit: Aloque a menor lacuna com a menor sobra; deve pesquisar a lista inteira, a menos que ordenado por tamanho (lento, varre a memória inteira)
 - Resulta em um o menor buraco restante
 - Worst-fit: Alocar o maior espaço; também deve pesquisar a lista inteira
 - Produz o maior espaço não alocado
- First-fit e best-fit são melhores do que worst-fit em termos de velocidade e utilização do armazenamento

Fragmentação

- **Fragmentação externa** – Existe espaço total de memória para satisfazer uma solicitação, mas não é contíguo.
- **Fragmentação interna** – A memória alocada pode ser um pouco maior do que a memória solicitada;

A primeira análise de ajuste revela que, dados os N blocos alocados, 50% dos N blocos perdidos para a fragmentação

- 1/3 pode ser inutilizável -> regra de 50%

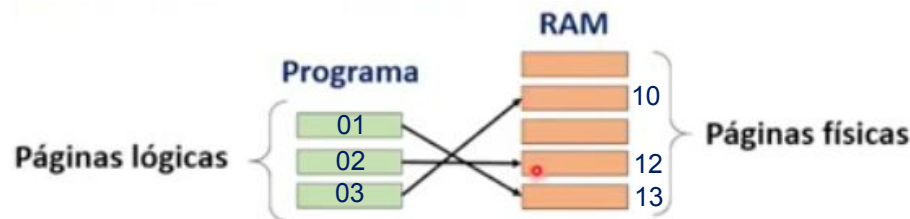
Como reduzir???

Fragmentação

- Reduzir a fragmentação externa por compactação
 - Embaralhe o conteúdo da memória para colocar toda a memória livre junta em um grande bloco!
 - A compactação só é possível se a realocação for dinâmica e for feita no momento da execução
 - Problema de E/S (por exemplo, lendo ou gravando no disco), não pode ser movido.
Operação trava na memória enquanto ela está envolvida na E/S
 - Fazer E/S somente em buffers do sistema operacional: pois os buffers são gerenciados separadamente e podem ser realocados de forma segura
- Obs: Agora considere que o armazenamento de discos e backup tem os mesmos problemas de fragmentação.

Paginação - Conceito

- Técnica de gerenciamento de memória, com divisões na memória física e nos processos.
- Cada página de um processo pode ser colocada em qualquer quadro livre da memória.
- As páginas não precisam estar juntas na memória física, melhora o aproveitamento da RAM



Memória Lóg	Memória Fis
01	13
02	12
03	10

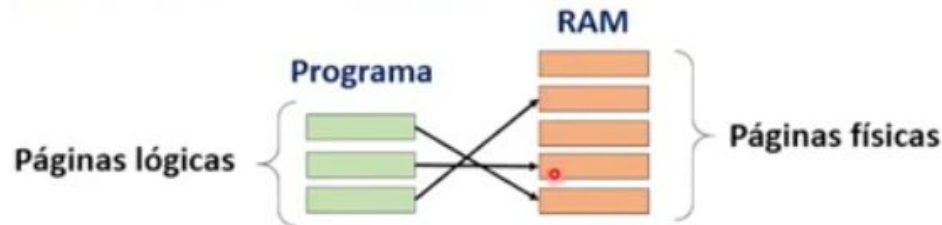
Paginação

- A compactação reorganiza a memória em tempo de execução para juntar todos os blocos livres em um único espaço contínuo.
 - Evita a fragmentação externa;
 - Evita o problema de blocos de memória de tamanho variável;
 - Ajuda a rodar processos maiores do que a memória disponível;
- Divide a memória lógica de um processo em blocos de tamanho fixo chamados páginas e a memória física (RAM) em blocos de mesmo tamanho chamados frames ou quadros.
 - Tamanho é potência de 2, entre 512 bytes e 1GB (4KB - 16 Mbytes).

Paginação

Características:

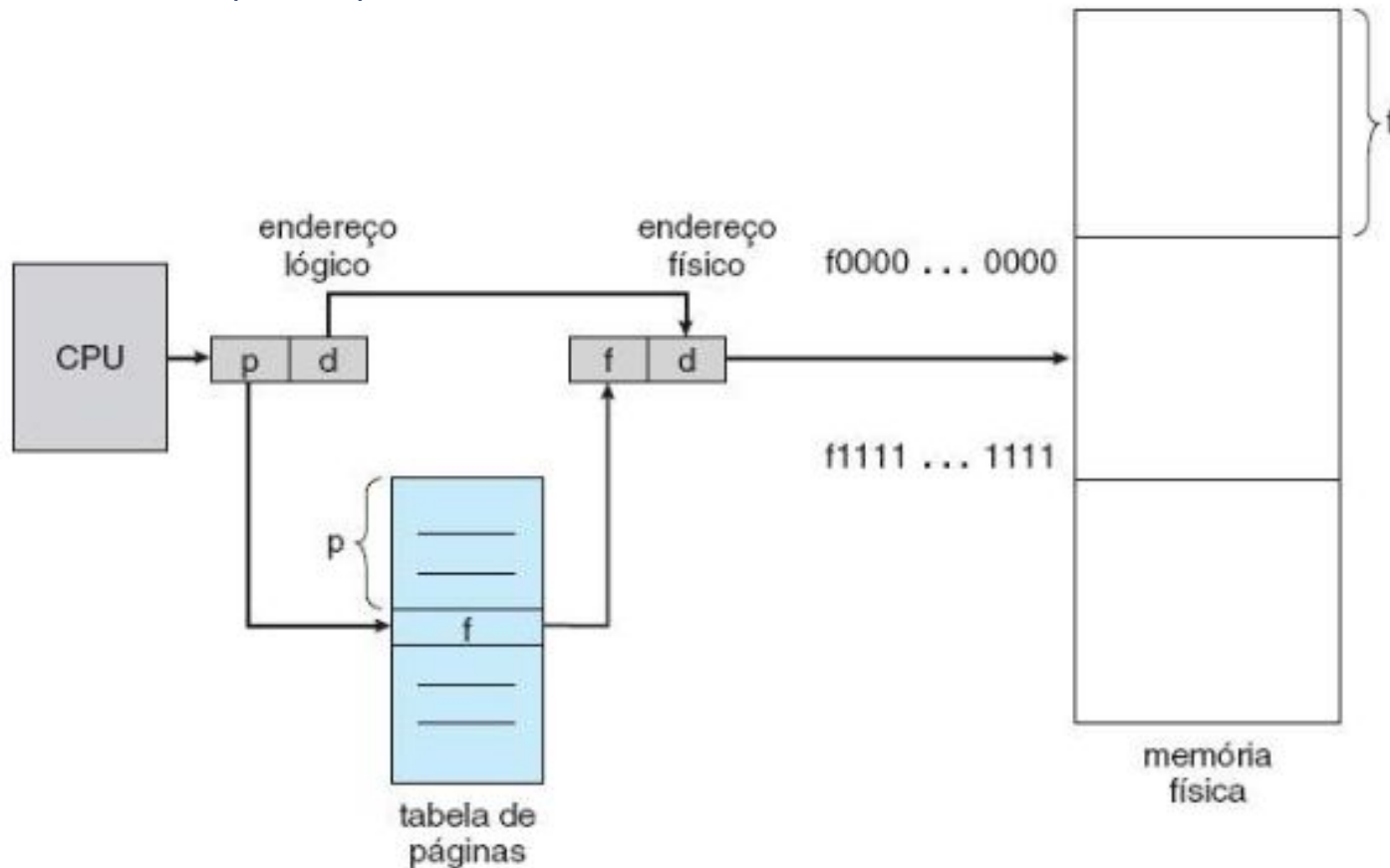
- Acompanhar todos os quadros livres.
- Para executar um programa de tamanho N páginas, precisa encontrar N quadros livres e carregar o programa.
- Configurar uma tabela de páginas para converter endereços lógicos em físicos



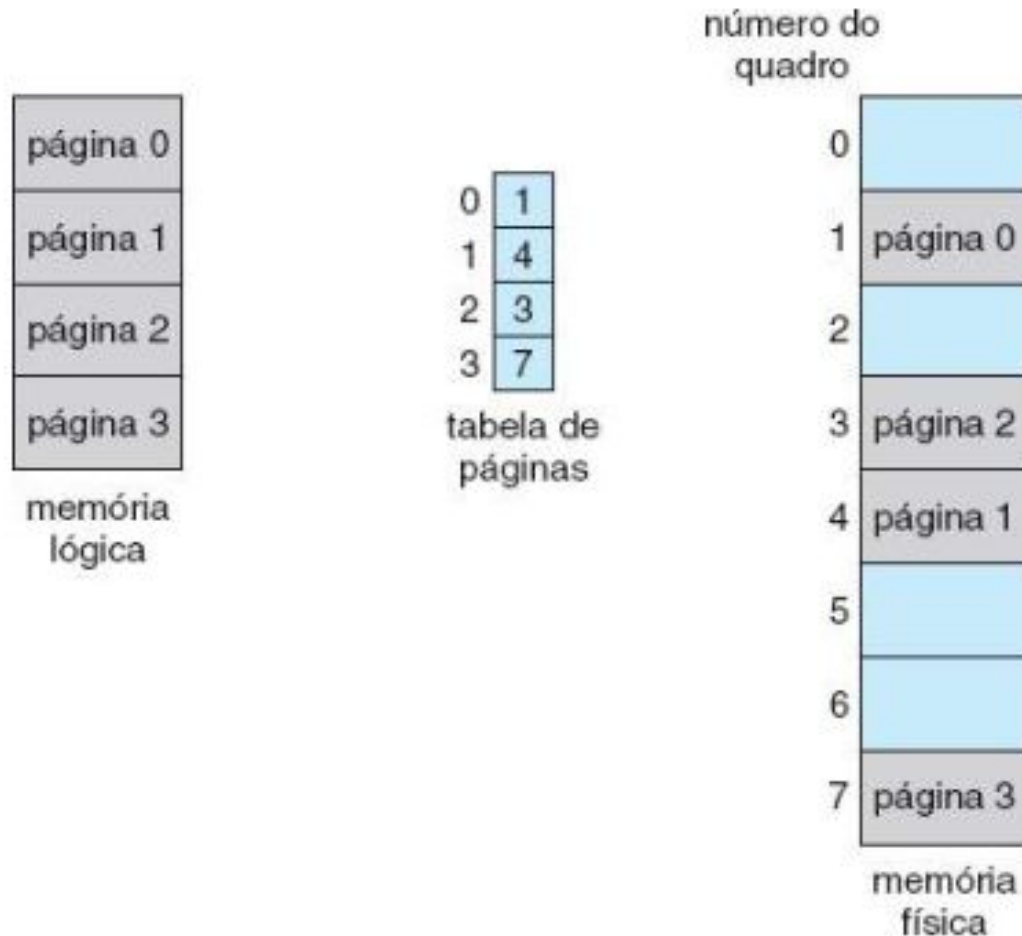
- O armazenamento de backup também é dividido em páginas, mas ainda temos fragmentação interna

Hardware de paginação

(p) – Índice da tabela de página
 (d) - Deslocamento (offset)



Modelo de Paginação das Memória Lógica e Física



Exemplo de paginação

End. lógico

- Offset 2 bits
- Usando um tamanho de página de 4 bytes
- Memória física de 32 bytes (8 páginas)

0	a
1	b
2	c
3	d
4	e
5	f
6	g
7	h
8	i
9	j
10	k
11	l
12	m
13	n
14	o
15	p

memória lógica

0	5
1	6
2	1
3	2

tabela de páginas

0	
4	i
	j
	k
	l
8	m
	n
	o
	p
12	
16	
20	a
	b
	c
	d
24	e
	f
	g
	h
28	

memória física

$$\text{End_físico} = \text{TabelaPaginas}[p].\text{Célula} * \text{TamPag} + d$$

Paginação – Calculando a fragmentação interna

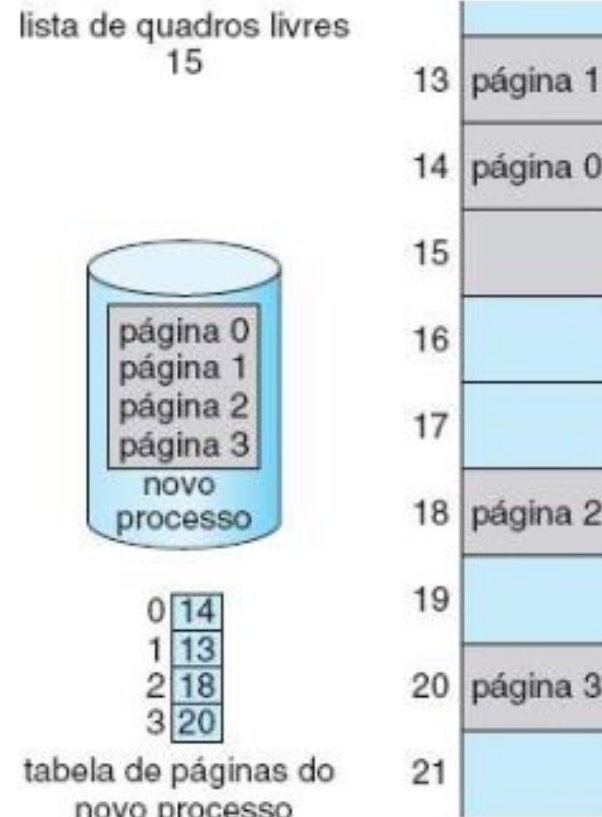
- Tamanho da página = 2.048 bytes
Tamanho do processo = 72.766 bytes
35 páginas + 1.086 bytes
Fragmentação interna de $2.048 - 1.086 = 962$ bytes
Fragmentação no pior caso = 1 quadro – 1 byte
 - Nesse exemplo, 2.047 bytes
- Em média, fragmentação interna perde por processo = 50% do tamanho da página.
- Tamanhos de quadro estão pequenos o suficiente?
 - Quadros muito pequenos: diminui a fragmentação, mas exigem muitas entradas na tabela de páginas (o que consome memória)
Os tamanhos das páginas crescem ao longo do tempo

Quadros (Frames) Livres



(a)

Antes da
alocação



(b)

Após a
alocação

Implementação da tabela de páginas

- A tabela de páginas é mantida na memória principal
 - Page-table base register (PTBR) aponta para a tabela de páginas.
 - Page-table length register (PTLR) indica o tamanho da tabela de páginas.
- Nesse esquema, todo acesso a dados/instruções requer dois acessos à memória
 - Um para a tabela de páginas e outro para os dados/instrução
- O problema de dois acessos a memórias pode ser resolvido com o uso de um cache de hardware especial de pesquisa rápida chamado *Translation Look-Aside Buffers (TLBs)* (também chamada de *memória associativa*)

Translation Look-Aside Buffer

- Memória especial: armazena temporariamente as traduções de endereços virtuais para físicos.
-
- TLBs são caches pequenas de hardware usadas para acelerar a tradução de endereços virtuais para físicos
- Alguns TLBs armazenam identificadores de espaço de endereço:
 - (Address Space Identifiers)- Diferenciar os processos.
 - Permite que o sistema proteja a memória de cada processo e evita que o cache precise ser "limpo" (esvaziado) toda vez que há uma troca de tarefa (troca de contexto).

Translation Look-Aside Buffer

- TLBs tipicamente pequenas (64 a 1024 entradas)
- Em uma miss de TLB, o valor é carregado na TLB para acesso mais rápido na próxima vez
 - Quando a “tradução” não é encontrada no cache, o valor é buscado na tabela de páginas (na memória principal) e carregado no TLB para que o acesso seja mais rápido nas próximas vezes.
 - Algumas políticas de substituição devem ser consideradas, por exemplo: o sistema precisa de regras para decidir qual tradução antiga remover para dar lugar a uma nova.

Como funcionam as Políticas de Substituição?

Como o TLB é muito pequeno (64 a 1024 entradas), ele enche rápido. Para colocar uma tradução nova, precisamos expulsar uma antiga.

- Memória associativa – pesquisa paralela

aponta para o início da tabela de páginas

Page #	Frame #

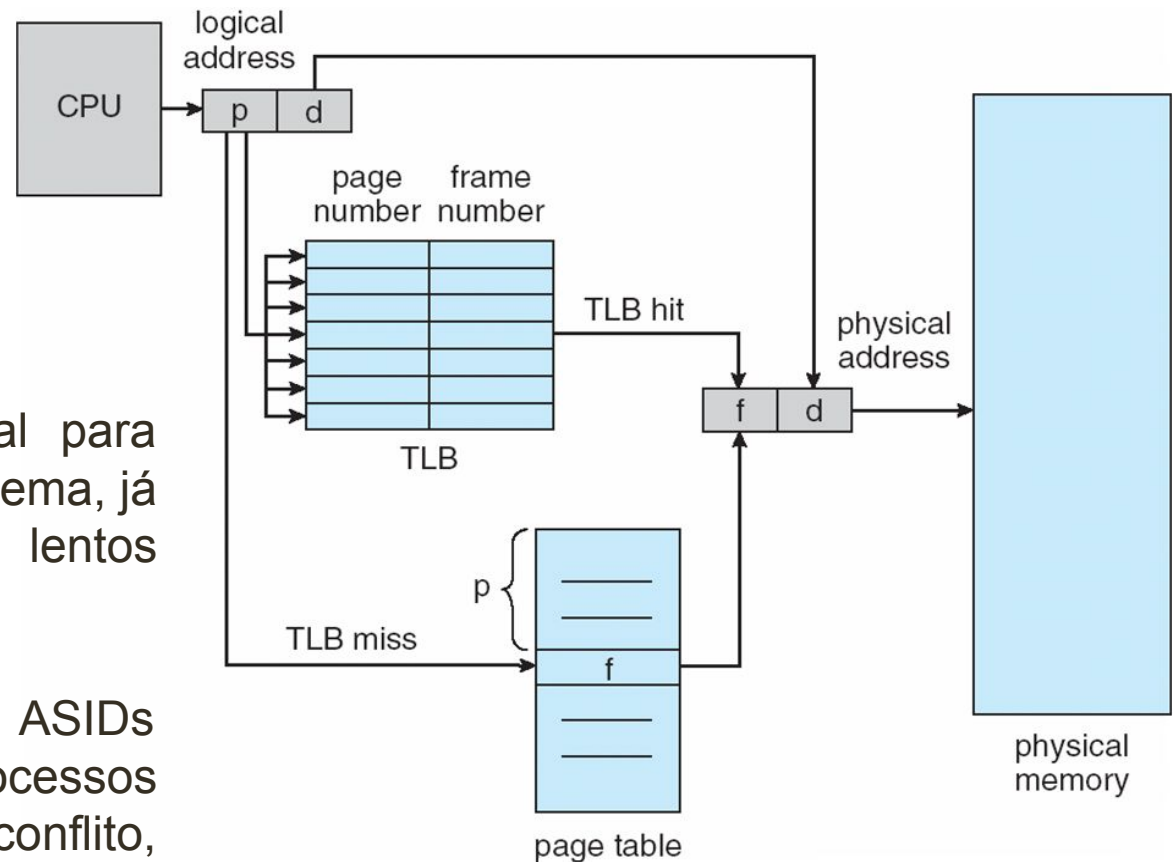
Tamanho da tabela de páginas

- Tradução de endereço (p, d)
 - Se 'p' estiver no registrador associativo, obtenha o quadro # da memória física
 - Caso contrário, obtenha o quadro # da tabela de páginas na memória (dois acessos)

Hardware de paginação com TLB

O uso de TLBs é essencial para aumentar a velocidade do sistema, já que acessos à RAM são lentos comparados à TLB.

Em sistemas multitarefa, os ASIDs permitem que múltiplos processos usem a TLB sem conflito, melhorando ainda mais a eficiência



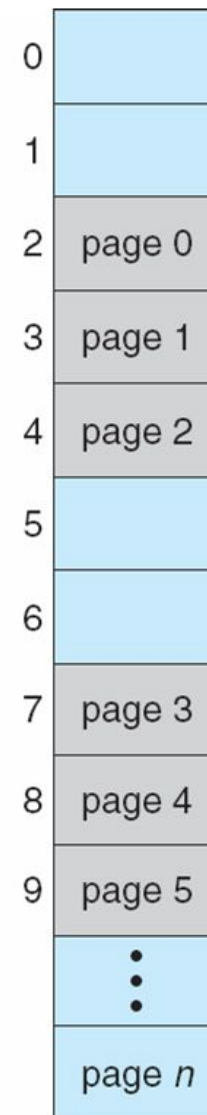
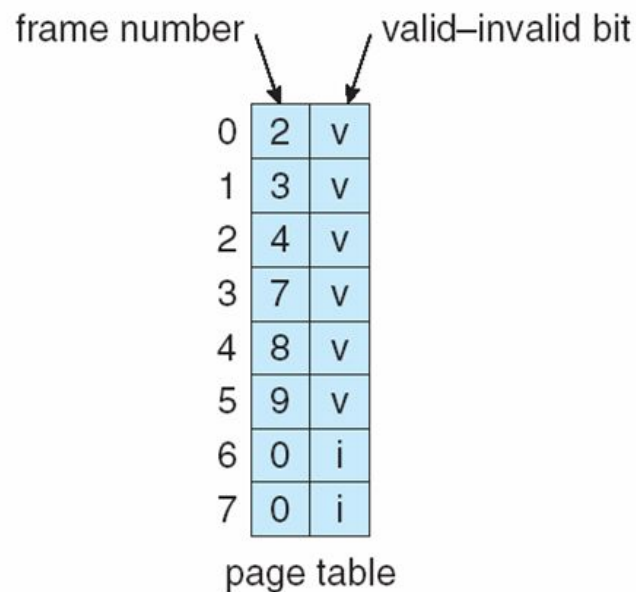
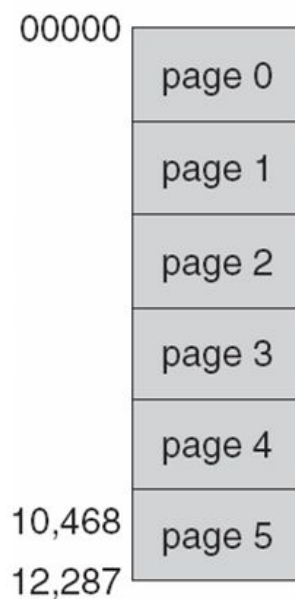
Tempo de Acesso Efetivo

- Taxa de acertos – porcentagem de vezes que um número de página é encontrado na TLB?
- Uma taxa de acerto de 80% significa que encontramos o número da página desejada na TLB 80% das vezes.
- Suponha que 10 nanossegundos para acessar a memória:
 - Se encontrarmos a página desejada na TLB, um acesso à memória mapeada leva 10 ns;
Caso contrário, precisamos de dois acessos à memória, por isso é 20 ns;
- Tempo de Acesso Efetivo (EAT)
$$\text{EAT} = (0,80 \times 10) + 0,20 \times 20 = 12 \text{ nanossegundos}$$
implicando 20% de lentidão no tempo de acesso
- Considere uma taxa de acerto mais realista de 99%
$$\text{EAT} = 0,99 \times 10 + 0,01 \times 20 = 10,1 \text{ ns}$$
implicando apenas 1% de desaceleração no tempo de acesso, já que o ideal (TEMPO MÉDIO) é 10ns.

Proteção de memória

- Proteção de memória: implementada associando o bit de proteção a cada quadro para indicar se o acesso somente leitura ou leitura/gravação é permitido
 - Também pode adicionar mais bits para indicar somente execução da página e assim por diante
- Bit válido-inválido anexado a cada entrada na tabela de páginas:
 - “válido” indica que a página associada está no espaço de endereço lógico do processo e, portanto, é uma página válida
 - “inválido” indica que a página não está no espaço de endereçamento lógico do processo
 - Ou usar page-table length register (PTLR)
- Quaisquer violações resultam em uma interrupção do tipo trap para o kernel

Bit válido (v) ou inválido (i) em uma tabela de página



Páginas Compartilhadas

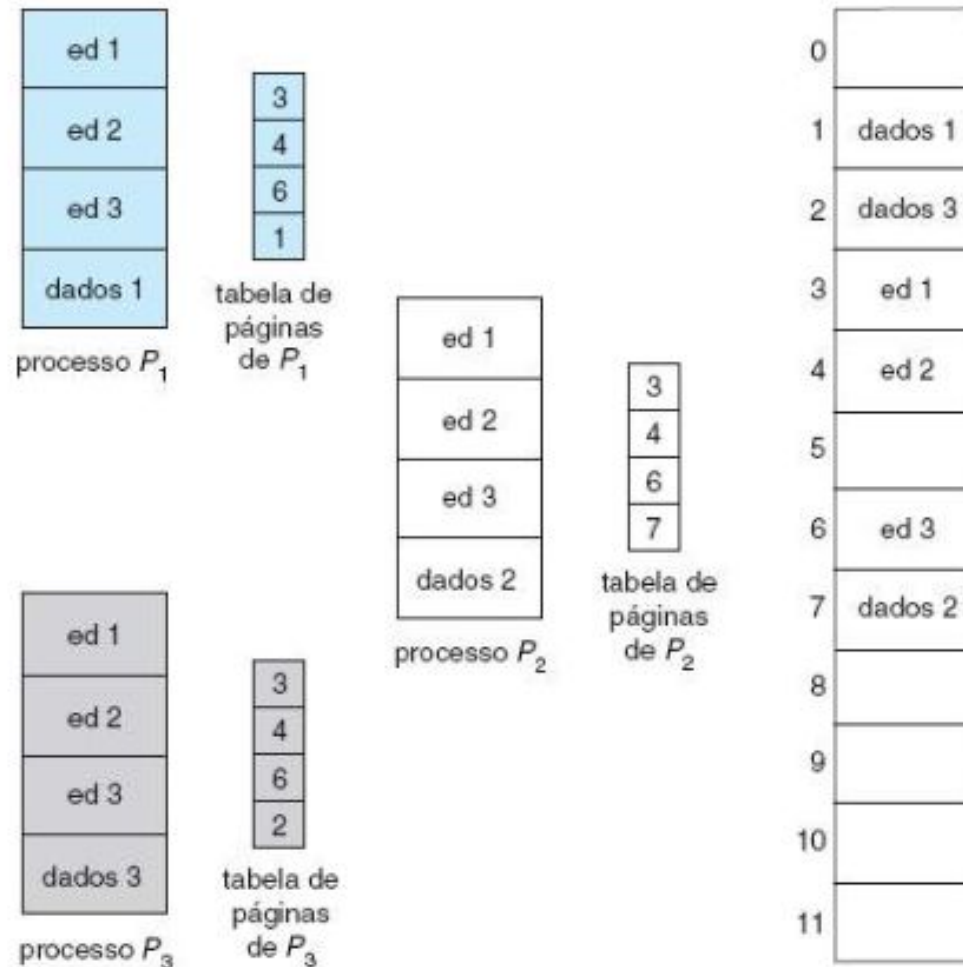
- Código compartilhado

- Uma cópia de código somente leitura compartilhada entre processos (ou seja, editores de texto, compiladores)
- Semelhante a vários threads compartilhando o mesmo espaço de processo
- Também útil para comunicação entre processos se o compartilhamento de páginas de leitura/gravação for permitido

- Código privado e dados

- Cada processo mantém uma cópia separada do código e dos dados
- As páginas para o código privado e os dados podem aparecer em qualquer lugar no espaço de endereço lógico

Exemplo de páginas compartilhadas



Estrutura da tabela de páginas

- As estruturas de memória para paginação podem ficar enormes usando métodos diretos
 - Considere um espaço de endereço lógico de 32 bits como em computadores modernos (2^{32})
 - Tamanho da página de 4 KB (2^{12})
 - A tabela de páginas teria $(2^{32}/2^{12}) = 2^{20}$ de entradas
 - Se cada entrada tiver 4 bytes $\rightarrow$ cada processo precisaria de 4 MB só para armazenar sua tabela de páginas
 - Ninguém reservar 4MB continuamente por processo na memória principal
 - A solução é dividir a tabela de páginas em unidades menores
 - **Paginação Hierárquica**
 - **Tabelas de página com hash**
 - **Tabelas de páginas invertidas**

Exercícios sobre Gerência de memória

Rodar o código no github Memory Cost:

No linux :

`main.cpp`
`stdafx.h`
`stdafx.cpp`
`targetver.h`



⚠ (Visual Studio / Windows)
⚠ (Visual Studio / Windows)
⚠ (Windows SDK)

Conteúdo:

- `main()`
- lógica do benchmark
- desempenho
- alocação (`new`)
- acesso à memória (`memset`, `leitura`)

Código Github - Memory Cost

Rodar o código no github e responder às seguintes questões:

1. Qual o tempo estimado para alocar 32MB na RAM?
2. Se rodar novamente este tempo será o mesmo?
Este é um tempo real ou virtual? Por que?
3. Qual seção da pilha o gerenciador de memoria atua neste ponto?
4. Qual o tempo estimado para escrever na memória?
5. Qual o tempo estimado para leitura na memória?
6. Quais fatores impactam na leitura para que seja mais rápida que a escrita ?

Código Github - Memory Cost

7. Qual o tempo total da manipulação?
8. Quanto tempo para desalocar a memória e atualizar a estrutura da heap?
9. A variavel SUM possui qual valor , por que?
10. Este valor é sempre o mesmo?
11. Por que mesmo sem escrever (desalocando) temos custo?
12. Qual a conclusão sobre Alocação de memoria e Acesso, dentro deste cenário?
13. Crie uma tabela com as principais operações/chamadas no código e faça uma relação custo e motivo.

Código Github - Memory Cost

Operação	Custo	Motivo
New		
Delete		
Write		
Read		
Alloc + write		
Alloc + read		

Código Github - Memory Cost

14. Altere o tamanho do buffer de 32MB para 64MB, 128MB e 256 MB faça uma tabela comparativa dos tempos de escrita e leitura.

Tamanho (MB)	Write (s)	Read (s)
32	0.1009	0.0855
64	?	?
128	?	?
256	?	?



UNIVALI

Universidade do Vale do Itajaí
Escola Politécnica

Fim